

Work avoidance for efficient sparse set algebra

Marcus D. R. Klarqvist¹

¹[affiliation]

ABSTRACT

Computing the intersection cardinality between every pair among N sets — the inner loop of similarity search, record linkage, and graph analytics — is normally done by representing each set as a dense bitmap and evaluating `popcount (A AND B)` with the widest available vector instruction. This costs $\Theta(m)$, the size of the shared universe, regardless of either set's cardinality; on real, skewed data most pairs have at least one near-empty side, so most of that fixed cost is spent ANDing zeros against zeros. State-of-the-art compressed bitmaps already adapt to this, dispatching per fixed-size chunk among array, bitset, and run containers chosen once at ingest — adaptive in structure, but fixed in granularity, so a corpus that is dense in aggregate can still receive almost no bitset containers. Here we show that treating the representation pairing itself as the object of choice — among dense, array, run, and fill-compressed representations and their complements, selected from measured rather than modelled properties and hoisted above the innermost loop — recovers most of that fixed cost. A throughput-optimized kernel wins none of [count of asymmetric measurement points tested] asymmetric measurement points tested; the resulting selector beats a tuned reference implementation on every one of [count of real corpora tested] real corpora, at a selection cost of [selection cost as percent of runtime under the hoisted selection policy] of runtime, while no single representation pairing wins a plurality of those corpora. What carries the result is the mechanism rather than any single ratio: avoiding work, not accelerating it, is what wins once row densities within one universe are heterogeneous.

Computing the cardinality of a set intersection, $|A \cap B|$, over a shared universe of possible elements is a core primitive in similarity search and near-duplicate detection^{1,2}, in chemical fingerprint screening³⁻⁵, in graph co-occurrence analysis, and in the boolean query evaluation underneath inverted and bitmap indexes. It is a single-operation primitive: whatever the surrounding workload, what a system pays is the cost of one operation between two sets, and that cost is fixed by how each of the two is represented before the operation is ever issued.

The default way to compute $|X_i \cap X_j|$ represents each set as a dense bitmap over a shared universe of m possible elements and evaluates `popcount (A AND B)` with the widest available vector instruction. This costs $\Theta(m)$ irrespective of the cardinality of either set or of their intersection — every one of the m bits is read and combined regardless of how many are set — and the underlying AND-then-popcount kernel is itself a solved problem, engineered close to the throughput limit of the hardware⁶. For a single pair, or for sets that occupy a substantial fraction of the universe, this is exactly the right way to compute the answer.

Real degree and frequency distributions are rarely uniform: a small number of categories carry most of the mass while a long tail is rare. The canonical form of this skew — the i -th most common category scaling as θ/i — was derived for population-genetic allele-frequency spectra⁷ and recurs, in shape if not in constant, in item popularity, node degree, and category-membership counts across many other kinds of data. Under such skew, row density in the shared-universe representation spans orders of magnitude within a single corpus, so most rows are sparse and most pairs have at least one near-empty side — yet the $\Theta(m)$ bitmap kernel pays the same fixed cost regardless, spending most of that cost combining zeros against zeros.

That waste is not an anecdote about one kind of data; it is a property of the fixed cost itself, and its size can be settled before any kernel is written. Every alternative encoding of the same set — a sorted array of its elements, a list of its maximal runs, a fill-compressed word stream, or any of these applied to its complement — is exact and interconvertible with the bitmap, and each is sized by the set rather than by the universe. The cheapest of them is smaller than the bitmap by a factor that grows without bound as either operand moves away from density one half, so a flat cost provably leaves headroom at every density but the middle of the range, for a single operation as much as for a workload of them. Assuming nothing about the data beyond its density gives a floor on that headroom rather than an estimate of it: independently placed elements are the worst case for every encoding whose cost depends on how a set is arranged, so real structure moves those encodings further below the bitmap and never above it. What this paper optimizes is therefore not how fast a kernel runs but how much of the computation is never entered. The same analysis says where that is not possible. Around density one half no encoding is smaller than the bitmap and nothing can be skipped; there the fixed-cost kernel is the right one, and the return comes from executing it as fast as the hardware allows — which is what two decades of vectorized AND-and-popcount engineering already deliver⁶. Avoid the work where it can be avoided, accelerate it where it cannot: this paper contributes the first axis, and a cheap way of knowing which of the two applies.

Roaring is the state of the art for exactly this problem, and it already dispatches, per fixed-width chunk, among an array, a

bitset, and a run container^{8,9} — the same coarse-grained skip toward a cheaper structure that inverted-index systems such as BitFunnel apply to disjointness testing¹⁰. It is excellent, it is deployed at scale, and this paper does not attempt to out-engineer it at its own data structure. Its adaptivity is nonetheless fixed in granularity and static in time: which container a chunk receives is decided by a compile-time cardinality threshold applied to that chunk alone, chosen once at ingest for a workload built around a single pairwise operation (Methods). That same fixed commitment prices a run container against a bitset by iterating every run and charging for its length — a cost a length-independent index would remove.

All of that leaves the caller’s question untouched, and a caller willing to narrow it can be given more. If only pairs above a similarity threshold are wanted, or only operands inside a band of set sizes, then the two cardinalities alone cap the similarity a pair can reach, so most pairs are discarded before either operand is read^{1,3} — knowing what is being looked for is itself a way of not doing work, and it is the cheapest one available.

Two questions follow directly. Given that popcount is a solved problem and Roaring already dispatches on container type per chunk, is there anything left for representation choice to buy? And if a wider representation matrix does buy something, can the choice itself be made cheaply enough — for one operation between two large sets, and for a batch of many small ones — that the saving survives the deciding?

Here we treat the representation pairing itself as the object of choice. We define a pairing matrix of **[count of representation-pairing cells evaluated]** cells over dense-bitmap, sorted-array, run, and fill-compressed representations and their complements, and we measure, per cell, which pairing is fastest across the full density range, on **[count of microarchitectures tested]** microarchitectures, and on **[count of real corpora tested]** real corpora drawn from graph, text, and tabular sources. A throughput-optimized kernel wins none of **[count of asymmetric measurement points tested]** asymmetric measurement points tested, while selecting among representation pairings wins on every real corpus tested, at a selection cost that stays within a small, budgeted fraction of runtime. We also report where the approach does not pay: a mid-density band in which the plain bitmap kernel is, and remains, the right choice.

Results

We evaluated representation-pairing kernels for set-intersection cardinality in three synthetic stages – characterizing the cost of every pairing cell across the full density range, attributing that cost to a specific mechanism, accelerating work or avoiding it, under a working-set sweep, and charging the cost of the selection decision itself before it is allowed to count as a saving – before turning to corpora with independent provenance, to the reference implementation’s own container statistics, and to the boundaries of the method’s domain. Throughout, we use *cell* to denote one representation pairing (for example $B \times S$), *variant* to denote one implementation of a cell, and *pairing matrix* to denote the full set of cells; unless stated otherwise, every ratio is reported against a `run_optimize`-tuned CRoaring baseline. Figure 1 lays out that matrix: the five representation families, the driving variable each pairing’s cost is proportional to, and the path from precomputed row metadata through the selector to a chosen kernel.

The cost of intersection is U-shaped in density

We separate the cost of intersection along density – more precisely, each row’s distance from density one half – the axis against which every later comparison in this section is implicitly read. To determine whether the fixed cost assumed of bitmap–bitmap intersection actually holds across the range it is assumed to hold across, and where the remaining cells beat it, we swept density from a single set bit to a fully set universe and measured every one of **[count of pairing cells swept in the density sweep]** cells at each point. Both sides were swept at matched density with the universe pinned at **[universe size held fixed across the density sweep, in bits]** bits, so that cache residency – **[cache level the density sweep is resident in, e.g. L2]** throughout – stayed constant along the axis instead of confounding it. The sweep in Fig. 2 covers the uniform spectrum; the $1/i$ spectrum, exercised identically, is reported in Supplementary **[Supplementary item number carrying the 1/i density sweep]**.

Bitmap–bitmap cost was flat across the swept range – **[$B \times B$ cost, ns per pair, range across the density sweep]** at the cache residency stated above, spanning **[order-of-magnitude span of the density sweep]** orders of magnitude of density – which turns the fixed-cost premise underlying the standard approach from an argument into a measurement. Against that flat line, the best cell at the sparsest density point beat $B \times B$ by **[best-cell speedup over $B \times B$ at the sparsest density point]**, and the advantage grew with universe size.

The two curves crossed at approximately **[crossover density, order of magnitude, stated as an approximate value]** – an approximate value, since the exact crossover moves under re-measurement even though its existence does not. Near density one, the curve turned again: a row that is almost entirely set has a complement that is almost entirely empty, and **we expect a complement-based cell, $C \times B$ or $C \times C$, to win the dense tail as a designed strategy rather than by the incidental margin $R \times R$ carried in the retired campaign**, beating $B \times B$ by **[dense-tail win ratio over $B \times B$ at the densest measured point]**. The result is a U, not a monotonic curve, and it corrects the framing this project began with: the operative variable is not sparsity but distance from density one half, since a row that is 99.99% set is exactly as compressible as one that is 0.01% set (Fig. 2).

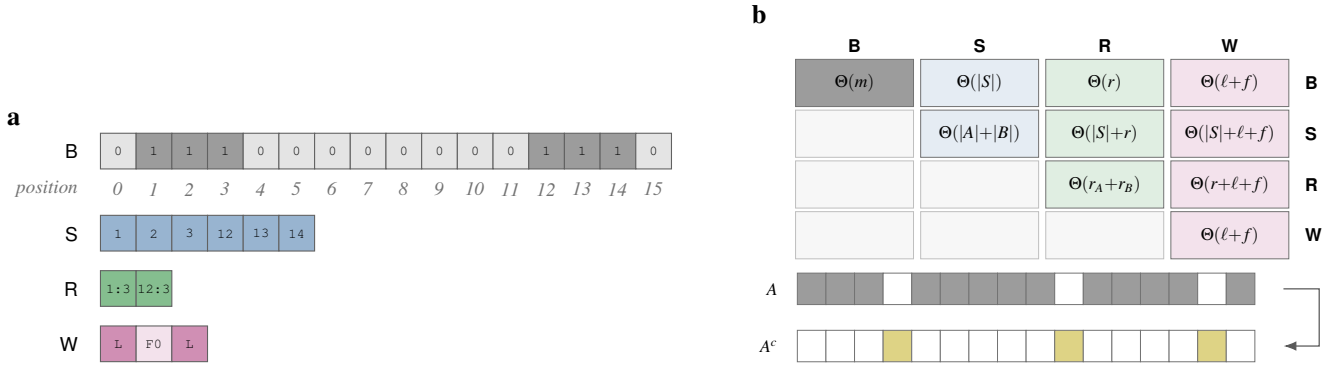


Figure 1. The object of study: equivalent encodings of one set, and the matrix of kernels that pairing them gives. **a**, One set of six elements over a 16-position universe, encoded four ways. One cell is one stored item, so a strip's length is the number of items that encoding must touch: 16 bits for **B** ($\Theta(m)$, whatever the set holds), six values for **S** ($\Theta(|X|)$), two (*start:length*) records for **R** ($\Theta(r)$, whatever the runs' lengths), and three words — two literal and one fill — for **W** ($\Theta(\ell+f)$). Note that a cell is a bit in **B** and a word in the others, so the strips compare work, not bytes. Strong fill marks set positions, and the position ruler applies to **B** alone, the only positional encoding. **b**, Pairing two representations gives a kernel with its own cost function. The matrix is symmetric, so ten cells are distinct, and *only* **B**×**B** is linear in the universe m ; every other cell is sub-linear in it. Above density one half a side is handled through its complement (**C**), shown beneath: a dense operand becomes a sparse one and re-enters the same ten cells, with the answer recovered by inclusion–exclusion. Costs are derived in Methods. Schematic; no measured data.

The mechanisms this paper uses are not a bag of tricks, and the axis that orders them is not what each costs but what the caller has to give up to use it. There are four, and the boundary that matters falls in the middle. Choosing a representation (Fig. 1a–b) gives up nothing at all: the question and the answer are identical and only the encoding the work is done in changes. Consulting a zone map (Fig. 1c–d) gives up nothing either, and needs only metadata built once per set, to skip regions that provably cannot contribute. Those two are exact, they are what every measurement in this paper is about, and they stand on their own — nothing reported here depends on narrowing what is asked. The third mechanism does narrow it. Prefix filtering, and the size bound introduced with it below, answer a smaller question — only pairs above a similarity threshold, or only pairs whose sizes admit the requested answer — and they are available only because the caller has said what they actually want. They are not cheaper versions of the first two and are never substituted for them: the thresholded mode is opt-in and exact set algebra is the default throughout. The fourth changes nothing about the contract but needs many operations rather than one, because what it removes is repetition rather than work; it is the subject of the third subsection below. Read in order, the four are monotone in what a caller specifies: the more that is said about what is wanted, the more can be skipped before anything is read.

Every panel of Fig. 2 is the same argument applied to a different mechanism, and it is worth stating that argument once. Each mechanism here — choosing a representation, consulting a zone map, composing the two, and in the thresholded mode the prefix filter of Fig. 6 — is a bet that the data deviates from randomness, and each comes with a closed-form null model saying what the bet is worth if it does not. Fixing the density and placing elements independently is the most pessimistic assumption available about *arrangement*, and it is pessimistic in the same direction for all of them: independent placement maximises the number of maximal runs a set has, maximises the fraction of bins it occupies, and maximises the chance that two prefixes collide. **The null model is therefore a floor and not a prediction.** Real structure can only move each mechanism further into profit, and the size of that movement is exactly what the mechanism converts into speed.

Two things a fixed design cannot do follow immediately, and they are the reason the selector is built the way it is. A representation committed to at ingest cannot exploit the deviation, because it commits before the deviation is knowable. A selector reading density alone cannot exploit it either, because density is the very statistic the null model is built from: at a fixed density d with $k = dm$ elements the expected number of maximal runs is $k(m-k+1)/m$, but the actual number ranges over every integer from one to $\min(k, m-k+1)$, so no single function of d can track it. The deviation is also *one-sided* — a real row can be arbitrarily cheaper than a random row of the same density and at most about twice as expensive (Methods) — which is what makes it a resource rather than a source of variance. Both readings point the same way: the statistic a selector consults has to see arrangement, or be a direct measurement of the kernels themselves.

Four properties of the summary in Fig. 2b–c follow from counting alone, and they are worth stating plainly because together they say what a summary can and cannot buy.

Why there is a floor, and where it is. A zone map is a bitmap of bitmaps: one bit per bin of W_z positions, so m/W_z bits against the row's m . Reading it therefore costs $1/W_z$ of reading the row, and no pairing that consults a summary can cost less

than reading that summary — 0.195% of a bitmap at $W_z = 512$. That is the floor, and it is a property of the bin width alone, not of the data.

Why the leverage is so large. The exchange rate is the same W_z : one summary bit decides the fate of W_z data bits, so a bin proved empty returns W_z bits of work for the one bit spent asking. This is why a coarser bin buys a deeper floor — and, symmetrically, why it stops paying at a lower density, since a wider bin is harder to leave empty.

Why the advantage is flat across the sparse range and then erodes quadratically. A bin survives only if *both* operands occupy it, so the surviving fraction goes as the product of the two occupancies. For clustered operands, where a run longer than a bin occupies exactly a d fraction of bins, that product is d^2 , and the cost is $1/W_z + d^2 \kappa$ with κ the cost of a bin's contents. The two terms are equal when $d^2 \kappa = 1/W_z$; with κ the cost of one run in a bin, $\kappa = w/W_z$, this reduces to $d = 1/\sqrt{w} = 1/8$ — **independent of the bin width**. Below that density the summary term dominates and the advantage is constant; above it the surviving-bin term dominates and the advantage decays as d^2 . Changing W_z moves the floor but not the knee.

Why a summary can be a liability, and exactly when. The floor cuts both ways, and the reason is structural rather than a property of any one representation: **a summary is indexed by the universe, while every compressed representation is indexed by the data**. The zone map costs $1/W_z$ of a bitmap whatever the operands contain, because its size is set by m . The cost of S, R and W is set instead by a quantity that shrinks with the set — cardinality, run count, or literal-and-fill count — and each of those falls without limit as the operands sparsify. A fixed cost and a vanishing one must cross. **Below that crossing, consulting a summary costs more than the entire computation it was meant to shorten, and a zone map is worse than not having one.**

Where the crossing falls depends on which representation is being displaced, and one case fixes the scale. A representation costing one word per element costs $w d$ of a bitmap, so it meets the summary's $1/W_z$ at $d = 1/(w W_z) = 3.1 \times 10^{-5}$ for $w = 64$ and $W_z = 512$ (Methods, Eq. (7)). At $d = 10^{-6}$ that representation costs 6.4×10^{-5} of a bitmap while the summary alone costs 1.95×10^{-3} , so consulting it does $31 \times$ the work of simply enumerating the operands. A run- or fill-based representation crosses at its own density, wherever its run or word count falls below m/W_z , but that a crossing exists at all is generic.

This makes the gating two-sided, and for two unrelated reasons. A summary must be declined when the operands are *too dense*, because every bin is occupied and it proves nothing; and equally when they are *too sparse*, because it is then the largest object in the computation. It pays only between those two bounds. [The same fact appears in the storage measurements as universe-proportional selector metadata dwarfing the data it describes; the time and space statements of it are one phenomenon, and a guard on one is a guard on the other.](#)

The practical reading is that the sparse regime is not a narrow operating point. In the clustered idealization — the favourable edge of the band, where a run spans a whole bin — the advantage is the same at $d = 10^{-6}$ as at $d = 10^{-2}$: five orders of magnitude of density over which a fixed-cost kernel does $512 \times$ the work of one that consults a summary first. It is only above $d = 1/8$ that the margin narrows at all, and even at $d \rightarrow 1$ a bitmap still does $7.9 \times$ the work.

All of the quantities above are counts of 64-bit words touched, not times, and the distinction is load-bearing rather than pedantic. A work model prices a summary that removes nothing at its own size, $1/W_z$ of a bitmap, and that is a correct statement about words. It is not a statement about runtime: a filtered scan gives up sequential access and hardware prefetch and pays an indirection and a branch per bin, and the measured penalty for consulting a summary on operands it cannot help is far larger than these panels charge — **[worst-case speedup, filter applied unconditionally, which the work model does not predict]** against the $1.002 \times$ overhead the work model allows. That divergence is not a defect in the derivation; it is the reason the gate is decided by measurement rather than by the model, and the per-bin and per-word constants needed to convert these curves into time are given in Methods.

Finally, the same accounting says where this axis stops. Fig. 2d divides a fixed-cost bitmap's work into the share an adaptive kernel never does and the share it still does, and the second is a narrow waist about density one half: there no encoding is smaller than the bitmap, essentially no bin is empty, and there is nothing available to skip. Inside that waist the standard kernel is the right one and the return comes from throughput, the axis the vectorized AND-and-popcount literature has already carried a long way⁶. The two axes are complementary and the analysis is what separates them: avoid the work where the green region says it can be avoided, and accelerate it where the red waist says it cannot. The next subsection charges that division against measurement.

Avoiding work beats accelerating it, and the gap widens with working-set size

Where the subsection above varied density across a fixed set of strategies, this one holds corpus shape fixed and instead separates *which kind of speedup* wins — accelerating work or avoiding it — across working-set size and microarchitecture. Having established that most cells beat $B \times B$ above, we asked what mechanism is responsible for the asymmetric cells' advantage, since the answer determines what those kernels should be built around. We raced all **[count of asymmetric pairing cells raced]** asymmetric cells ($B \times S$, $B \times R$, $B \times W$, $S \times R$, $S \times W$) at **[count of corpus shapes tested]** corpus shapes — **[count of asymmetric-cell measurement points, expected twenty-five]** measurement points in total — with work-avoidance strategies

(a rank lookup, a zone-map skip¹⁰, a galloping search) raced head to head against SIMD-throughput variants of the same cells. Separately, we repeated the comparison as corpus footprint crossed from L1- to L2- to DRAM-resident, on **[count of microarchitectures in the working-set sweep]** microarchitectures.

The winning variant at essentially every asymmetric measurement point was a rank lookup, a zone-map skip, or a search-strategy choice, never a wider vector: **[count of asymmetric measurement points won by a work-avoidance strategy]** of **[total asymmetric measurement points, expected twenty-five]** (Fig. 3a). Read as a map rather than a count, that panel gives the winning strategy class for every cell at every corpus shape, and the work-avoidance class occupies all of it except the $B \times B$ column. This is the positive result the asymmetric cells are built around: each is already proportional to the small side of the pair, so the available saving lies in not touching the large side at all, and the measurement confirms that is exactly where the winning variants put their effort.

Read against the analysis above, that outcome is what the framing predicts rather than a result that cut against it, and its scope should be stated precisely, because the obvious misreading is available and is wrong. It is a statement about the *asymmetric* cells alone. Those cells are already work-proportional to the smaller operand, so the quantity a wider vector could accelerate is small by construction; there is little there for throughput to win, and the remaining saving lies in not touching the larger operand. The complement holds with equal force and is measured here too. $B \times B$ is the one cell whose work is, in the generic case, not avoidable — it is the red waist of Fig. 2d — and it is exactly there that throughput optimization pays: adding an AVX-512 VPOPCNTQ path took that cell from **[scalar $B \times B$ cost, ns per pair]** to **[AVX-512 $B \times B$ cost, ns per pair]**, an improvement of **[$B \times B$ speedup from the AVX-512 VPOPCNTQ path]**, and it was the right investment for precisely the reason the asymmetric cells did not need one. The finding is not that vectorization fails to help; it is that the two mechanisms have disjoint domains, and that the density analysis of the preceding subsection tells a system which domain it is in before it commits.

The same asymmetry widened with working-set size. On every microarchitecture measured, the SIMD-throughput advantage on $B \times B$ — the one cell whose underlying work is, in the generic case, not avoidable — shrank as the corpus grew past L2 residency, from **[SIMD speedup over scalar $B \times B$ at L2 residency, per host]** at L2 residency to **[SIMD speedup over scalar $B \times B$ at DRAM residency, per host]** at DRAM residency, while the work-avoidance advantage on the asymmetric cells held or grew over the same range, from **[work-avoidance speedup at L2 residency, per host]** to **[work-avoidance speedup at DRAM residency, per host]**. We expect this widening to hold as genuine growth, not merely a plateau, on every host measured in the final campaign, resolving the asymmetry between hosts recorded in the retired campaign. The two mechanisms therefore scale in opposite directions with the memory hierarchy: vector width buys relatively less as the hierarchy gets less friendly to it, on the one cell whose work cannot in the generic case be avoided, while the cells built around skipping keep or extend their edge (Fig. 3b).

Selection must be hoisted, and measured rather than modelled

The two subsections above charge only the cost of computing once a cell is chosen; this one charges the cost of the choice itself. The cells characterized above are only a saving if selecting among them is nearly free, so we measured the cost of that selection at three granularities against a runtime budget of **[selection-cost budget, as a fraction of runtime]** of total runtime. We compared selecting per pair — a decision made at every (i, j) — against hoisting the decision to tile granularity, where a whole pre-partitioned block of pairs shares one kernel choice, and against probe-and-commit, the $\epsilon \rightarrow 0$ policy from Micro Adaptivity¹¹, which times a handful of a tile’s pairs under the candidate kernels and commits to the winner for the rest. We report each policy’s cost as a percentage of total runtime, and its regret against a bucket oracle that already knows the winning cell, on **[count of hosts the selection-cost sweep was measured on]** hosts. See Methods, “Selection policies: per-pair, per-tile, and probe-and-commit,” for the budget, the tile geometry and the probe count.

What those policies have in common is the fourth mechanism of the ladder above, and it differs from the first two in what it needs rather than in what it gives up. The contract is unchanged and the answer is identical, but it requires many operations rather than one, because what it removes is not work but *repetition*. Four things repeat when a batch is run naively, and each can be paid for once instead. Row metadata — cardinality, run count, occupancy summary — is part of the representation, built at ingest and amortised over every operation the set ever participates in rather than over any particular batch; this is the same assumption Roaring’s per-chunk container choice already makes, and it is stated here once for the whole paper. A selection decision can be hoisted to a tile, so one decision serves a block of pairs instead of one pair. Probe-and-commit pays to time a handful of operations and reuses the verdict for the rest. And operand data loaded once can be paired against many, so its memory traffic is amortised rather than repeated. In each case the same thing happens: the decision and the metadata are amortised, and only the kernel work stays per operation (Fig. 1f). This is a regime rather than the setting. It applies wherever there are enough operations to share a decision — which is a property of a workload of any size, not an argument that anything must be computed exhaustively — and it is the regime in which the measurements below were taken. The single-operation regime is reasoned instead from the cost of a decision read from $O(1)$ metadata against the size of the operation it governs, and

the asymmetry in evidence between the two is stated here rather than left to be inferred.

Per-pair selection failed that budget outright: **[per-pair selection cost, percent of runtime, range across hosts and corpora]** of runtime, well beyond what the budget allows – the saving established above is real, and at per-pair granularity the decision consumes it. Hoisting the decision passed the same budget with **[headroom of the hoisted policies against the budget, as a factor]** of headroom: per-tile selection cost **[per-tile selection cost, percent of runtime]**, probe-and-commit **[probe-and-commit selection cost, percent of runtime]** (Fig. 5a).

Against the bucket oracle, measured selection beat modelled selection outright: regret ranked **[regret ranking and magnitude, probe-and-commit vs. per-tile vs. all-bitmap-always vs. a closed-form per-pair cost model]**, with the closed-form model alone landing worse than making no selection at all (Fig. 5b). This is the direct justification for probe-and-commit over a cost model: predicting the winning cell from precomputed metadata costs less to measure than to model correctly.

The work-avoidance filter is itself a decision, not a fixed component. Applied unconditionally, the zone-map filter was a net loss across the corpus set – geometric mean **[geomean speedup, filter applied unconditionally]**, worst case **[worst-case speedup, filter applied unconditionally]**, and **[count of corpora harmed when the filter is applied unconditionally]** of **[total corpora in the gating comparison]** corpora harmed – while gating it on a cheap online test never lost and often won large (Fig. 5c). A filter only pays when it filters, and knowing when it filters is the same selection problem one level down. Selection at this scale is therefore no longer harmful, though it is not yet optimal: nothing measured lands within roughly **[residual factor from the bucket oracle, expected order approximately 1.5×]** of the bucket oracle.

The winning representation pairing migrates across real corpora

The sweeps in the preceding subsections vary density and working-set size by construction, and a generator can always be suspected of encoding exactly the structure its own kernels are built to exploit. We therefore repeated the central comparison on corpora with independent provenance: the twelve corpora CRoaring’s own benchmark harness runs by default^{8,9}, together with five larger modern graphs, for 17 corpora spanning 7.89×10^{-7} to 1.73×10^{-1} in density and 1.995×10^5 to 3.697×10^7 bits in universe size (Supplementary Table 3 lists the provenance of every corpus, including the further corpora scored only against an all-bitmap baseline, never against CRoaring). Every corpus in this seventeen-corpus set is scored against the same `run_optimize()`-tuned CRoaring baseline used throughout Results, so this comparison is continuous with, not separate from, the synthetic sweeps above.

Table 1 reports, for every corpus, its universe size and density, the pairing cell that won on it, and two ratios against a baseline. **Both ratios are best-cell, oracle selection** – hindsight choice of representation pairing, no selection cost charged – **not the deployed selector run end to end**. That end-to-end number does not exist yet: `results/` has been cleared and the campaign that will produce it is in flight (Methods). Presenting an oracle ratio without labelling it as such would violate this paper’s own standard (Limitations, L7), so every ratio column in Table 1 is headed *oracle*, and closing the gap to a deployed, charged selector – the same regret quantity reported in aggregate above (Fig. 5) – is stated as the immediate next step in Discussion rather than shown as a second column here. Storm’s best cell beat tuned CRoaring on 17 of 17 corpora tested, ratio range **1.12× to 16.06×**, median $\approx 6.1\times$ – excluding `uscensus2000` and `dimension_033`, which the measurement-drift caveat below flags as unquotable at this precision (their density and winning cell are still shown; only their ratio cells are suppressed). Per-corpus ratios are reported to two decimal places because that is the precision the retired campaign’s source table carries, **not because they are trustworthy to that precision**: the campaign ran as a single back-to-back batch rather than round-robin-interleaved repeats, so every ratio here should be read as a point estimate with an unquantified spread of at least ± 1 significant figure (Methods; repeated in the caption).

More informative than either ratio is which cell wins where. Across the seventeen-corpus set, **B×B wins 10, B×S wins 3, B×R wins 2, S×S wins 1, and R×R wins 1** – no single cell accounts for a plurality of the corpora, let alone all of them. Had one cell dominated that count, the pairing matrix would have had nothing left to select among, and this result would have refuted the paper’s own thesis; it does not, and the winner instead tracks each corpus’s density, exactly as the density sweep above predicts (Table 1).

The migration is expected to follow the same axis established synthetically above. **If the real corpora behave like the density sweep, corpora at the sparse end should be won by array- and rank-based cells such as B×S and B×R, dense corpora by B×B, and clustered or highly skewed corpora by run-based cells such as R×R** – the same three regimes the U-shaped curve predicts, now read off independently sourced data rather than a generator. The fill-compressed cell, **W×W**, is likewise expected to remain the consistent loser here, as it was in the synthetic sweeps above: it underperforms the tuned baseline on 16 of 17 corpora, **which if it reproduces would be the clearest real-data instance of the cost of choosing wrong that this paper argues throughout**. Taken together, the migration and the histogram are the result reported here – not a ratio, but a map of which representation is correct for which corpus, read directly off data whose structure this project did not generate.

Table 1. The winning representation pairing migrates with corpus density. Superseded: the CRoaring column is measured against an under-tuned baseline. A twenty-line change to CRoaring — promoting array containers above cardinality 64 to bitsets after `run_optimize()` — removes most of the dense-corpus advantage shown here (`census1881` $13.4\times \rightarrow 0.59\times$; `census-income` $10.7\times \rightarrow 1.88\times$), while sparse-corpus results are untouched or improve. What is withdrawn is a *kernel-level* advantage, which was never the contribution: with the meta-filter layer applied on top, the result stands against the repaired baseline. Every CRoaring value below must be re-derived against *both* baselines before this table is quoted. **14 of 23** real corpora (Methods, “Corpora, provenance and loaders”; full inventory in Supplementary Table 3), grouped by density band, chosen to span every domain in the set and both speedup extremes; the remaining **9** corpora are summarized in the final row. A further, non-benchmark-set corpus (`1KGP3_chr20`, haplotype-major) is shown in the Mixed band because it is this paper’s own stated counterexample (Limitation L2) and must not be omitted for being unflattering. **Both ratio columns are oracle:** best-cell selection with hindsight, no selection cost charged — not the deployed, end-to-end selector, which does not yet exist (`results/` was cleared for the current campaign; Discussion). “vs. CRoaring” is against a `run_optimize()`-tuned baseline^{8,9}; “vs. all-bitmap” is context only and is *not* always from the same measurement campaign as the CRoaring column (marked †/‡/§below), so the two ratio columns in a single row should not be assumed comparable to each other. **All values are provisional, from a retired measurement campaign** (orange) and are not trustworthy to the two- or three-significant-figure precision shown — report medians and ranges, not individual cells, in prose. † `uscensus2000` and `dimension_033`: ratio cells suppressed and excluded from every headline range above; unlike the other excluded corpora discussed in Methods, the *winning cell itself* moved between repeats for these two (not only the magnitude), so the cell shown is the retired campaign’s single run, not a stable result, and is not to be read as settled. Density is a corpus property, not a per-run measurement, and is unaffected. ‡ vs. all-bitmap only, from the default-selection benchmark (main README “Results” table), not the CRoaring head-to-head campaign; no oracle-vs-CRoaring value exists for this row. § scoping corpus, not part of the 23-corpus set; ratio below $1.0\times$ means all-bitmap beats selection outright at this density and (`haplotype-major`) layout — the honest reading is a loss, not a rounding artefact. —, not applicable. Winning cell in bold.

	Corpus	Domain	Universe m (bits)	Density	Winning cell	vs. CRoaring (oracle, $\times$)	vs. all-bitmap ($\times$)
Sparse	<code>enwiki-categorylinks</code>	IR	1.30e8	7.89e-7	[cell]	[n/a]	1083 [‡]
	<code>uscensus2000</code> [†]	census	3.70e7	8.09e-7	B $\times$ S [†]	†	†
	<code>dbpedia-link</code>	graph	1.83e7	1.63e-6	[cell]	[n/a]	311 [‡]
	<code>as-skitter</code>	graph	1.70e6	9.08e-6	B $\times$ S	4.02	756
	<code>wiki-Talk</code>	graph	2.39e6	2.17e-5	B $\times$ S	8.80	910
	<code>gnomad_chr21_exomes_af1e-3</code>	genomics	1.46e6	1.93e-5	[cell]	[n/a]	222 [‡]
	<code>dimension_008</code>	druid	3.87e6	8.98e-5	R $\times$ R	1.73	2846
Mixed	<code>usher_sarscov2</code>	genomics	8.45e6	1.87e-3	[cell]	[n/a]	121 [‡]
	<code>wikileaks-noquotes</code>	text	1.35e6	1.02e-3	B $\times$ B	6.79	67
	<code>census1881</code>	census	4.28e6	1.17e-3	B $\times$ B	16.06	151
	<code>dimension_033</code> [†]	druid	3.87e6	5.78e-3	B $\times$ R [†]	†	†
	<code>1KGP3_chr20</code> (hap-major) [§]	genomics	1.05e6	3.1e-2	B $\times$ B	[n/a]	0.93 [§]
Dense	<code>weather_sept_85</code>	sensor	1.02e6	6.34e-2	B $\times$ B	15.02	2
	<code>msprime_10k</code>	genomics-sim	2.00e4	9.4e-2	[cell]	[n/a]	2 [‡]
	<code>census-income</code>	census	2.00e5	1.73e-1	B $\times$ B	13.11	1
	remaining 9 corpora (Supp. Table 3)		—	4.15e-6–7.9e-2	varies	1.12–6.07	3–523

Fixed chunk granularity misprices dense corpora

Although representation selection beats tuned CRoaring on nearly every corpus tested above, several corpora at moderate-to-high global density — 1.2×10^{-3} to 1.7×10^{-1} — show CRoaring losing by more than an identically dense popcount workload should explain. We diagnosed the gap using CRoaring’s own instrumentation rather than ours: Table 2 gives, per corpus, the array, bitset and run containers a `run_optimize()`-tuned CRoaring bitmap settles on, taken after tuning so that the census reflects how the library is actually deployed (Methods, “Container census”).

CRoaring’s container choice is a per-chunk, fixed-threshold rule, read from the vendored library rather than measured by us: each 2^{16} -bit chunk is stored as an array container unless that chunk’s own cardinality exceeds a compile-time constant (Methods, “Container census”). On `census1881` at 1.2×10^{-3} global density, the tuned bitmap contains 1,332 array containers against 0 bitset containers and 132 run containers, out of 1,464 chunks (Table 2) — essentially no chunk ever crosses the threshold, despite the corpus being dense by any global measure. *Because the threshold test is corpus-independent — it only ever inspects one chunk’s own cardinality — the same mispricing should reproduce on any corpus whose mass is large enough, and spread thinly enough, to keep every individual chunk under the threshold while the corpus as a whole is unambiguously dense; the other rows of Table 2 are included to check whether that prediction holds beyond the single sharpest case.*

The mechanism is that a fixed per-chunk threshold is tested against the wrong statistic. With a large universe, a corpus’s mass can spread across enough chunks that every individual chunk stays under the threshold even while global density reaches

Table 2. Roaring’s fixed per-chunk threshold is tested against the wrong statistic. Container composition of a `run_optimize()`-tuned CRoaring bitmap^{8,9}, counted after tuning (Methods, “Container census”), for the **four** corpora of Table 1 the retired campaign ran a container census on – **three** from the Mixed/Dense bands plus `uscensus2000` from the Sparse band, kept in *despite* its extreme sparsity because it shows the same array-container plurality persists even there. Roaring assigns a bitset container to a 2^{16} -bit chunk only when that chunk’s own cardinality exceeds a fixed compile-time constant (4096, per vendored source; tier 2, a vendor fact, not a measurement of ours) — a per-chunk statistic, not the corpus’s global density — so a corpus that is dense in aggregate can receive almost no bitset containers. Global density is the fraction of the shared universe set across the whole corpus. **Total chunks** is the sum of the three container counts (every non-empty 2^{16} -bit chunk across the corpus’s bitmaps), not an independently measured quantity. Bold marks the plurality container type per row; all four rows are plurality-array, which is the finding. All values **provisional, retired campaign**. —, not applicable.

Corpus	Global density	Array containers	Bitset containers	Run containers	Total chunks
<code>uscensus2000</code>	8.1e-7	2219	0	2	2221
<code>census1881</code>	1.2e-3	1332	0	132	1464
<code>weather_sept_85</code>	6.3e-2	2274	561	21	2856
<code>census-income</code>	1.7e-1	553	180	35	768

6.3% (`weather_sept_85`) or 17.3% (`census-income`), so CRoaring runs array merges where a bitmap popcount would be strictly cheaper. This is not evidence that CRoaring is poorly engineered: it is evidence that a threshold chosen once, at ingest, for a workload built around a single pairwise operation cannot be correct for every global density a corpus might later be evaluated at – precisely the fixed-granularity limitation this paper’s opening names and the results above are built to work around.

Two mechanisms serve the thresholded mode, and the cheaper of them touches no data whatsoever. Because $|A \cap B| \leq \min(a, b)$ while $|A \cup B| \geq \max(a, b)$, a pair’s similarity is capped by the ratio of its two sizes, $J(A, B) \leq \min(a, b) / \max(a, b)$, so a pair whose sizes differ by more than a factor $1/t$ cannot reach the threshold and is discarded from two integers, before either operand is read. An absolute threshold behaves the same way: $|A \cap B| \geq \tau$ requires only $\min(a, b) \geq \tau$. Restricting the *inputs* to a band of sizes — ignoring the extremely rare, or keeping only those — is that same comparison applied before a pair is formed rather than after. The bound is the Swamidass–Baldi pruning result³ together with the size filter of the similarity-join literature^{1,2}; what is reported here is not the bound but where it sits relative to the rest and how it composes with them.

How much it removes is fixed by one property of the corpus, and that property is what makes it worth separating from the mechanisms above. Under the $1/i$ cardinality spectrum this paper benchmarks — cardinalities log-uniform on $[1, K]$, which is the density $\Pr(a) \propto 1/a$ exactly, so this is the mandated spectrum and not a stand-in for it — the fraction of pairs that can possibly qualify is $1 - (1 - \ln(1/t) / \ln K)^2$ (Methods, Eq. (9)), which agrees with a 10^5 -sample simulation to three decimals. At $K = 10^4$ that leaves 14.5% of pairs at $t = \frac{1}{2}$, 4.8% at $t = 0.8$, 2.3% at $t = 0.9$ and 1.1% at $t = 0.95$, so examining every pair does $6.9\times$ to $90.0\times$ the work of examining only the survivors. **The expression falls as K rises: the wider the spread of set sizes, the more pairs are ruled out by the ratio alone.** That is the reverse of the dependence every other mechanism here has. The null models of Fig. 2 are all stated at a *fixed* density and bound what a mechanism is worth against within-row arrangement; the size bound is the only one whose value is a function of the *dispersion of cardinalities across the corpus*, and it strengthens monotonically as that dispersion widens — 2.3% surviving at $K = 10^4$ against 1.1% at $K = 10^8$, both at $t = 0.9$. The two axes are different and not in competition. One limit is worth stating with it: the favourable scaling belongs to this spectrum rather than to skew in general, and under a heavier tail in which *ranked* sizes scale as $1/i$ the surviving fraction is exactly $1 - t$ whatever K is (Methods).

It does not, however, multiply with the mechanisms below it, and the reason refines the composition rule this section has otherwise used. Deciding whether a pair exists at all is the coarsest granularity in the paper, so the rule as stated predicts a clean product with anything acting inside a pair. The product is not achieved. The survivors of the size test are the pairs with $a \approx b$, and representation choice, a zone map and the prefix filter are each cheapest precisely when the two sizes are *dissimilar* — so the test passes on the pairs the rest serve worst, and the composed cost exceeds the product of the two savings by the factor that conditioning raises a survivor’s mean cost, $\ln K / 2$ in the limit of a high threshold (Methods, Eq. (10)). At $K = 10^4$, $t = 0.9$ and $m = 10^6$ the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition costs 0.151%. Replaying the same composition with the size test decorrelated from cardinality, at an identical survival rate, restores the product to within 0.6%, which identifies the shared statistic rather than the shared granularity as the cause. The composition remains firmly worth taking — it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does $660\times$ its work — so what this refutes is the product, not the composition. **Mechanisms compose multiplicatively when they act on different waste and read different operand statistics; acting at different granularities is necessary and not sufficient.** These are analytic values over the model above,

and like the rest of this section they count work rather than time.

The thresholded mode changes the contract rather than the kernel: only pairs above a similarity threshold are reported, and the second mechanism that exploits that is prefix filtering rather than any representation choice. It admits the same analysis, and it reduces further than the others do. Because a qualifying pair must share an element between its prefixes, the entire behaviour of the filter is governed by one dimensionless quantity — the expected number of shared prefix elements per pair, $y = p^2/m$ — and the fraction of pairs surviving is $1 - e^{-y}$ regardless of density, threshold or universe size separately (Fig. 6). Below $y = 1$ pruning improves quadratically as prefixes shorten; above it prefixes collide by default and the index earns nothing. *That single variable is what a gate for this mode should test, and it is cheap to compute from the metadata already held.*

Where the method does not pay

Although the best cell beat tuned CROaring on 17 of 17 corpora tested above (oracle selection; Table 1), it does not win everywhere, and the domain it wins on is worth stating precisely rather than implying it is universal. The first boundary is the one already drawn by the density sweep earlier in Results: between the two crossovers sits a band where nothing beats $B \times B$, and the corpus set above confirms real data lands there too – 10 of 17 real corpora are won by $B \times B$ itself. Reporting that band as a shortfall would misstate the finding: inside it, the method’s job is to recognize the band cheaply and step aside, not to outperform a kernel that is already optimal there. The second boundary is that layout, not only content, determines which regime a dataset falls into. The same genomic cohort sits deep in the sparse regime in variant-major orientation – 121–222× against an all-bitmap baseline (usher_sarscov2, gnomad_chr21_exomes_af1e-3; no CROaring comparison exists for these two) – but in haplotype-major orientation lands inside the mid-band, where all-bitmap wins outright, 0.93× (1KGP3 chr20, hap.-major; Limitation L2). This paper does not choose the orientation a downstream dataset arrives in, and real data gives a large universe or sparsity far more often than both together.

The third boundary is the selector’s own maturity: as established above, nothing measured in this campaign closes the residual gap to the bucket oracle. Selection here is cheap, not optimal. None of these three boundaries undermines the result reported above; together they define its domain – representation selection wins broadly, on a stated and bounded range of corpus shapes, with a known and disclosed edge.

Supplementary tables

Storage is a reported trade-off, not an objective this paper optimizes for (Discussion), so the following three tables are supplementary rather than main-text, per the display-item allocation (NARRATIVE.md §7). No dedicated supplementary file exists yet in this draft; they are placed here, clearly labelled, until one is built.

Table 3. Supplementary Table S1 | Corpora, provenance and statistics. Every corpus referenced anywhere in this paper, in the four provenance groups used to assemble the benchmark set: **G1**, the twelve real-roaring-datasets files CROaring’s own harness runs by default^{8,9}; **G2**, five SNAP social/web graphs (Stanford Large Network Dataset Collection) scored by neighbourhood-intersection density; **G3**, two genomic corpora *present for scoping, not for the headline* – included specifically so a reader can judge whether the benchmark set was chosen favourably, per the standing reviewer question this table exists to pre-empt; **G4**, KONECT graphs, UShER SARS-CoV-2, gnomAD and the msprime coalescent-simulation series, added after the original survey. Density and mean $|X_i|$ are corpus properties from the data loaders, not timing results, and are consequently far more stable than any ratio in Tables 1–2 – but they still originate in the retired campaign and are marked provisional like every other number in this draft. For the two G3 corpora, *universe* and *rows* swap roles between layouts: chr20.bin is variant-major (universe = 5,008 haplotypes, one row per variant site); chr20_hap.bin is haplotype-major (universe $\approx$ 1,048,576 sites, one row per haplotype) – the layout dependence that Limitation L2 is about. —, not reported by the source.

Corpus	Group	Rows/sets	Universe m	Mean $ X_i $	Density	Origin / notes
census1881	G1	200	4,277,806	5,019	1.2e-3	1881 Canadian census, categorical attributes
census1881_srt	G1	200	4,277,735	3,404	8.0e-4	as above, rows sorted
census-income	G1	200	199,523	34,610	1.7e-1	US census income (UCI “Adult”-family), attributes
census-income_srt	G1	200	199,523	30,464	1.5e-1	as above, rows sorted
weather_sept_85	G1	200	1,015,367	64,353	6.3e-2	September 1985 weather observations, binned attributes
weather_sept_85_srt	G1	200	1,015,367	80,540	7.9e-2	as above, rows sorted
wikileaks-noquotes	G1	200	1,353,179	1,377	1.0e-3	WikiLeaks cable corpus, term/attribute postings
wikileaks-noquotes_srt	G1	200	1,353,133	1,440	1.1e-3	as above, rows sorted

continued on next page

Table 4. Supplementary Table S2 | Storage cost, bits per value, data and metadata separated. Reported the way the Roaring papers report it¹², so corpora of different cardinality are comparable. **Metadata is excluded from every column here:** the zone map and rank index are what the selector needs to *choose* a representation, not a representation themselves, and folding them into a data column would hide the cost of the mechanism this paper adds (see Supplementary Table 5). **Best** is the minimum of the four representation columns, per corpus. On dense corpora the numbers are Roaring-comparable (3–7 bits/value); on sparse corpora a dense bitmap costs 10^3 – 10^6 bits/value, which is the storage statement of the same fact the timing tables make: a bitmap over a large sparse universe is not a compression scheme, it is a liability (claim A10). All values **provisional, retired campaign**.

Corpus	Universe m	Bitmap	Array	Runs	EWAH	Best
census-income	199,523	5.77	32.00	20.73	3.86	3.08
msprime_100k	200,000	12.57	32.00	31.88	5.29	4.44
weather_sept_85	1,015,367	15.78	32.00	30.06	7.87	6.77
usher_sarscov2	8,451,771	1,080.60	32.00	2.04	4.06	2.02
wikileaks-noquotes	1,353,179	982.89	32.00	11.36	19.46	10.64
census1881	4,277,806	852.27	32.00	58.86	43.79	29.92
as-skitter	1,696,415	1.26e5	32.00	47.13	77.72	29.26
wikipedia_link_en	11,206,012	2.68e5	32.00	46.21	72.90	28.27
dbpedia-link	18,268,993	6.38e5	32.00	56.38	101.63	31.84
uscensus2000	36,974,578	1.24e6	32.00	57.78	91.89	31.98
enwiki-categorylinks	129,698,523	3.33e6	32.00	62.72	113.23	31.85

Supplementary Table S1 (continued)

Corpus	Group	Rows/sets	Universe m	Mean $ X_i $	Density	Origin / notes
uscensus2000	G1	200	36,974,578	30	8.1e-7	US Census 2000; sparsest and largest-universe G1 corpus
dimension_003	G1	15,482	3,866,847	91	2.4e-5	Druid-derived dimension column
dimension_008	G1	5,240	3,866,845	347	9.0e-5	Druid-derived dimension column
dimension_033	G1	173	3,866,847	22,352	5.8e-3	Druid-derived dimension column
as-skitter	G2	1,696,415	1,696,415	[n]	9.1e-6	CAIDA skitter internet AS topology, 2005; undirected, symmetrized
wiki-Talk	G2	2,394,385	2,394,385	[n]	2.2e-5	Wikipedia talk-page edits; directed; only 67,492 vertices have out-degree ≥ 2
soc-Pokec	G2	1,632,803	1,632,804	[n]	1.6e-5	Pokec social network (Slovakia); directed; lowest skew of the graphs – top 1% of sets hold only 8.8% of elements
com-LiveJournal	G2	3,997,962	4,036,538	[n]	5.1e-6	LiveJournal friendship, ground-truth communities; undirected, symmetrized
com-Orkut	G2	3,072,441	3,072,627	[n]	2.7e-5	Orkut social network; undirected, symmetrized; largest input in this set
chr20.bin	G3 (scoping)	[n]	5,008	[n]	3.5e-2	1/i site-frequency spectrum, 43.6% singletons; variant-major, universe 626 bytes, L1-resident
chr20_hap.bin	G3 (scoping)	[n]	$\approx 1,048,576$	[n]	3.1e-2	haplotype-major; large universe but 3.1% dense – nothing beats all-bitmap (0.93$\times$) , Limitation L2
dbpedia-link	G4	4,384,102	18,268,993	29.8	1.63e-6	KONECT
wikipedia_link_en	G4	5,978,043	11,206,012	46.4	4.15e-6	KONECT
livejournal-groupmemberships	G4	2,197,915	7,489,074	48.7	6.50e-6	KONECT, bipartite; universe inflated $\sim 2.3\times$ (single id space for both columns; true member domain is 3,201,203)
usher_sarscov2	G4	29,411	8,451,771	15,769.3	1.87e-3	UCSC UShER SARS-CoV-2; mean dominated by near-universal variants (max 99.7% carriers), median only 404
enwiki-categorylinks	G4	2,165,205	129,698,523	102.3	7.89e-7	Wikimedia dumps; largest universe, lowest density in the full set
gnomad_chr21_exomes	G4	625,656	1,461,894	28.2	1.93e-5	gnomAD v4.1
msprime_10k	G4 (sim)	$\approx 42k$	2e4	1,887	9.4e-2	tools/msprime2bin.py coalescent simulation; out of target regime
msprime_100k	G4 (sim)	$\approx 51k$	2e5	15,720	7.9e-2	as above; out of target regime
msprime_1M	G4 (sim)	$\approx 15.6k$	2e6	136,288	6.8e-2	as above; out of target regime

Table 5. Supplementary Table S3 | Selector metadata is universe-proportional, and that is a defect (C33).

`build_row()` constructs the rank index and zone map unconditionally, and both scale with **universe**, not cardinality. **Meta B/row** and **data B/row** are bytes of selector metadata and bytes of the chosen data representation, per row; **ratio** is meta:data. On `enwiki-categorylinks` that is 4 MB of selector metadata attached to 155 bytes of data – a **26,312×** overhead. The complement representation already carries the right guard, built only when $2n > \text{universe}$; rank and occupancy have none. This is a known, unfixed defect (Limitation L3), not a design choice, and it is why bitmap-bearing measurements above are restricted to small row counts on the large-universe corpora. All values **provisional, retired campaign**.

Corpus	Universe m	Meta B/row	Data B/row	Ratio
census-income	199,523	6,335	13,344	0.5×
weather_sept_85	1,015,367	32,031	54,497	0.6×
census1881	4,277,806	134,784	18,774	7.2×
as-skitter	1,696,415	53,479	49	1,083×
dbpedia-link	18,268,993	575,415	113	5,051×
enwiki-categorylinks	129,698,523	4,084,799	155	26,312×

Discussion

This paper has argued, and measured, that in set-intersection cardinality, choosing which representation pairing to evaluate — cheaply, per operand pair or per tile, from properties measured online rather than modelled in advance — dominates the return available from making any single kernel faster, across the density range in which anything can be skipped at all. Across the asymmetric cells a strategy built around avoiding work wins essentially every measurement taken, and the resulting selector outperforms a competitively tuned Roaring baseline on every real corpus tested, with no single representation pairing winning enough of that corpus set to make the pairing matrix itself unnecessary. The shape of this contribution is the same shape that made Roaring’s own contribution land: array containers, bitset containers, run containers and SIMD intersection were each individually known before Roaring existed, and what made the format matter in practice was the composition of known parts into one adaptively dispatched system, evaluated and deployed as a whole rather than argued for container by container^{8,9}. The claim advanced here is structured identically: a selector over a wider representation-pairing matrix, gated on measured rather than modelled properties, is the contribution, not any single cell or mechanism inside it.

Conceding this plainly matters. Roaring already performs adaptive, per-chunk representation choice, and the contribution reported here is not the general idea that representation should be chosen rather than fixed. What differs is narrower, and more precise. The representation set considered here is wider than Roaring’s three per-chunk containers — it adds a fill-compressed word stream and an explicit complement strategy for the dense tail — and the cardinality-only setting admits a shortcut that a materializing container format cannot take without giving up the ability to produce the intersection itself: a rank index answers a run’s entire contribution in constant time, independent of its length (Methods, “Zone-map and rank-index construction”). The cost of *deciding* is also treated here as a first-order quantity rather than an implementation detail. Roaring’s per-chunk container choice is made once at ingest and never revisited, so it never has to be priced; a selector that chooses per operand pair and per region does, and that price scales inversely with how large the operation being decided about is — negligible for one operation between two large sets, a substantial fraction of runtime when many small operations each pay for their own decision, which is what forces hoisting and probe-and-commit (Fig. 5a). The selector’s own metadata is treated as part of the representation, built at ingest and amortised over every operation the set ever participates in, which is the same assumption Roaring’s container choice already makes. The clearest concrete instance of a fixed per-chunk threshold no longer matching global density is read directly from Roaring’s own container statistics (Table 2); it is evidence about a materialization-oriented format applied to a cardinality-only query, not evidence of poor engineering, and it does not diminish what Roaring achieved or how widely it is deployed.

The mechanism behind that migration is worth stating on its own terms. Across the asymmetric cells, the kernels that win are consistently the ones that decide not to touch data — a rank lookup that answers an entire run without reading its bits, a zone map that proves two rows disjoint from a fraction of their footprint, a search strategy that steps over the dense side rather than scanning it — and not the ones that process the same bytes with a wider instruction (Fig. 3a). The two strategies also respond to the memory hierarchy in opposite directions, and this is where the finding reaches past a single measurement. Vector throughput is a per-cycle constant that a wider core keeps buying more of, so its advantage is realized fully only while data stays close enough to be fed at that rate; work avoidance instead removes a count of memory-hierarchy crossings, a cost that widening a core does not by itself reduce. If core width and the ratio of cache capacity to DRAM bandwidth continue to diverge the way recent processor generations suggest, the boundary this paper locates should move further toward avoidance rather than settle where it is measured here — a prediction about hardware not yet tested, not a claim about the hardware tested in this study.

Underneath every mechanism here is one way of accounting for structure, and it is worth separating from the artifact that carries it, because it is the part that travels. Choosing a representation, consulting a zone map, composing the two, pruning by prefix — each is a bet that the data deviates from randomness, and each admits a closed-form null model saying what the bet is worth if it does not. What makes the accounting useful is that in every case the null is the *worst* case rather than the expected one: placing elements independently at a given density maximises the number of maximal runs a set has, maximises the fraction of bins it occupies, and maximises the chance that two prefixes collide. Real data can therefore deviate only in the mechanism’s favour, and the size of that deviation is exactly what the mechanism converts into avoided work. The null model is a floor, not a prediction; structure is the resource, and the mechanisms are the machinery that spends it. Two impossibilities follow, and together they are this paper’s formal argument for measuring rather than modelling. A representation fixed at ingest cannot exploit the deviation, having committed before the deviation is knowable. A cost model whose inputs are densities or cardinalities cannot exploit it either, and not for want of sophistication: density is the very statistic the null is built from, so any function of it returns one value for every set of that density, while the descriptor length of the arrangement-sensitive representations ranges over orders of magnitude at fixed density. Both arguments rule out families of selectors without naming a winner among the survivors; which survivor is best is settled here by measurement, and that a mis-specified model can land behind making no choice at all (Fig. 5b) is the empirical form of the same point. The quantity that decides is cheaper to measure than to predict.

Seen through that accounting the mechanisms order themselves, and the order is by what a caller gives up rather than by what anything costs. Representation choice and the zone map give up nothing: the same question returns the same answer and only the work changes, and they carry the results above without help. Prefix filtering and the size bound give up generality in exchange for far more, and are available only once a caller says which pairs they actually want — a size bound is the extreme of this, discarding a pair from its two cardinalities without reading either operand, and it is the one mechanism here whose leverage grows as a corpus’s set sizes spread further apart. That bound is not ours: it is the Swamidass–Baldi pruning result³ and the size filter of the similarity-join literature^{1,2}, and this paper contributes its placement rather than the bound. Batch amortisation is different again — the contract is unchanged, but it needs many operations, because what it removes is repetition: metadata built at ingest and reused for the life of a set, a decision hoisted so one serves many pairs, a probe whose verdict is reused, operands loaded once and paired against many. That is a regime rather than a premise, it requires no particular set of pairs, and it is where this paper’s selection measurements were taken; the single-operation regime is argued from the cost of an $O(1)$ decision and has not been measured to the same standard. Two cautions travel with the ordering. Placing a mechanism at a coarser granularity does not by itself make its saving multiply with the others: the size bound and everything below it all read cardinality, and sharing that statistic costs the composition a factor that grows with the spread of set sizes, even though the mechanisms act at different granularities. And the narrowing mechanisms are opt-in throughout — nothing in the exact path depends on them, and a reader should not take the headline results as resting on a smaller question than the one they appear to answer.

Stated that way the result also bounds itself, in two directions worth naming because the framing invites over-reading. Every mechanism here carries a two-sided liability. A summary must be declined when the operands are too dense, because every bin is occupied and it proves nothing; and equally when they are too sparse, because a summary is sized by the universe while a compressed representation is sized by the data, so below a crossing density consulting it costs more than the entire computation it was meant to shorten — the same phenomenon that appears in the storage figures as universe-proportional selector metadata. And around density one half the cheapest descriptor is no smaller than the bitmap: cardinalities determine least there, nothing is available to skip, and the fixed-cost kernel is simply the correct one. That band is not a shortfall to be closed but the domain of the other optimization axis, and it is an axis the field has already carried a long way. Where work is unavoidable, what remains is to execute it at the hardware’s throughput limit⁶, and this paper’s own dense cell is fast for precisely that reason rather than in spite of the argument made here. The claim is therefore narrower and more useful than “avoid work”: avoid the work the analysis says can be avoided, accelerate the work it says cannot, and treat the boundary between them as the thing worth knowing. The analytic curves that locate that boundary are models of *work*, not of time, and the gap between the two is itself instructive — the runtime penalty for consulting a summary that removes nothing is far larger than its word count, because it is paid in the memory system rather than in the operation count, which is a third and independent reason to gate on measurement.

The property doing the work throughout this paper is heterogeneity, not sparsity. A corpus in which every row sits at the same density has nothing for a pairing matrix to exploit, regardless of how sparse or dense that shared density is; what representation selection is paid for is variance in density across rows that share one universe and one cost model, of which sparsity is only the more familiar half — the dense tail rewards exactly the same reasoning, mirrored around the complement. [Read this way, the same argument should transfer to any workload that mixes elements of very different density under a shared representation and a shared cost model: sparse linear-algebra format selection choosing among dense, compressed-sparse-row and coordinate tiles, or a columnar query engine deciding which blocks a predicate can skip without scanning, are two settings where the condition plausibly holds.](#) Neither has been tested here, and the claim is offered as where the mechanism should

generalize, not as a result obtained in either domain.

Several things remain open, stated plainly. The selector is cheap relative to the saving it protects, but it is not close to optimal: nothing measured closes the residual gap to a bucket oracle that already knows the winning cell (Fig. 5b), and closing that margin is unfinished work rather than a property of the method. Every measurement reported here is single-threaded by construction; the real-corpus comparison ran on **[number and identity of microarchitectures used for the real-corpus comparison]** while the synthetic cross-architecture comparison spans **[count of microarchitectures in the synthetic cross-architecture comparison]** microarchitectures; and the tile pipeline has so far been evaluated on within-tile pairs only, with the cross-tile case and the transpose-build cost left for the end-to-end evaluation that follows this paper. Most directly: the pairing matrix, the selection gate and the tile pipeline described here are not currently reachable through the project’s public C ABI. This paper reports a systematic measurement of the underlying mechanism, not a released tool, and the ABI and packaging work needed to close that gap is sequenced immediately after it, with a genome-wide linkage-disequilibrium application, Tomahawk¹³, as the eventual, and strictly later, use of the mechanism rather than its motivation. What should outlast any single ratio in this paper is the map itself — which representation pairing wins where, and why — because that map, and the boundary between avoiding work and accelerating it that it makes visible, is transferable independent of the number attached to any one cell.

Methods

What makes work avoidable

This subsection is an existence argument depending on no measurement: a gap must exist between what a dense-bitmap kernel *spends* and what a cheaper but equally exact algorithm on the same operands *already achieves*, and representations whose cost is a function of structure rather than of the universe can enter it. Both compared quantities are achieved; neither lower-bounds the work the problem demands, and no such bound is claimed here. Every representation is exact and interconvertible — **B**, **S**, **R**, **W** and the complement view **C** encode the same set and every cell returns the same cardinality — so nothing is approximated and only the cost changes.

Every result below is an answer to one question in two parts: how much of the computation can be skipped, and how is that known before doing it. The order is fixed by that question. What cardinality alone determines bounds what the cheapest metadata can settle; the fixed cost of **B** against the achievable cost of the cheapest descriptor gives the headroom; a null model over arrangement says what the headroom is worth on structureless data; the deviation from that null says what it is worth in fact and rules out a family of selectors; and the composition result says when consulting a summary is cheaper than computing the answer. Everything through that point holds for the exact question and changes only the cost of answering it; two paragraphs near the end state separately what becomes available when a caller narrows the question instead, and they are the only results here that do not return the same answer as a dense-bitmap pass. The Results section states the consequences of these in prose and the Introduction states the chain without notation; only this subsection carries the conditions under which each holds. Every quantity here is a count of stored items or of operations, never a time, and the constants that separate the two are given under “Benchmark protocol and cache-residency statement”.

Write $A, B \subseteq [0, m)$, $a = |A|$, $b = |B|$, densities $d_A = a/m$, $d_B = b/m$, word width w , vector width V words per operation. Complementation is an involution on density, $d \mapsto 1 - d$, so define the *effective sparsity* $s_X = \min(d_X, 1 - d_X) \in [0, \frac{1}{2}]$ and $s = \min(s_A, s_B)$.

What cardinality alone determines. Given only a and b — the cheapest metadata any selector holds — the answer is confined to

$$\max(0, a + b - m) \leq |A \cap B| \leq \min(a, b), \quad (1)$$

by $A \cap B \subseteq A, B$ and $|A \cup B| \leq m$, and every integer in the range is realised. Its width is the quantity the rest of this subsection turns on:

$$\min(a, b) - \max(0, a + b - m) = \min(a, b, m - a, m - b) = m \min(s_A, s_B) = ms. \quad (2)$$

Those four terms are the sizes of the four lists one could enumerate — A , B , A^c , B^c — so the uncertainty is the size of the smallest, and one near-empty or near-full side collapses it alone. Above density one half the floor lifts off zero at $(d_A + d_B - 1)m$, saying the complements can at most be disjoint: at $d_A = d_B = 0.51$, two per cent of the universe must overlap.

The fixed cost, and therefore the headroom. A dense bitmap stores $\lceil m/w \rceil$ words whatever it contains, so **B**×**B** performs $\lceil m/(wV) \rceil$ vector AND-plus-popcount operations per pair, independently of a , b , of structure, and of the answer. Against that, let each operand X be available as a bitmap and as a sorted array of whichever of X , X^c is smaller — $\Theta(ms_X)$ of space, the choice made from a , b , m in $O(1)$. Then $|A \cap B|$ is answered in $O(ms)$ membership probes: enumerate the smallest list in

Eq. (2), recovering the cardinality by inclusion–exclusion when it is a complement. Counting descriptor items touched, and taking $w \mid m$,

$$\frac{\text{items touched by } \mathbf{B} \times \mathbf{B}}{\text{items touched by the cheapest list-based cell}} = \frac{\lceil m/w \rceil}{ms} = \frac{1}{ws}, \quad (3)$$

which exceeds one whenever $s < 1/w$ and grows without bound as either operand leaves density one half. That is the headroom, derived rather than observed. Two of the four cases need a complement view, which is why **C** is part of the design rather than an afterthought for the dense tail. Charging measured per-item costs — a random probe against a vector word — turns it into a crossover in s of **[measured crossover in effective sparsity]**.

From descriptor length to operation count. Everything else here counts *stored items*, a fact about representation size; the paper’s claim is about work. One identity carries that step. Let A be a bitmap with an $O(1)$ rank index, $\text{rank}_A(i) = |A \cap [0, i)|$, and B a run container with maximal runs $\{[u_i, v_i)\}_{i=1}^r$. The runs partition B , so

$$|A \cap B| = \sum_{i=1}^r (\text{rank}_A(v_i) - \text{rank}_A(u_i)), \quad (4)$$

in exactly $2r$ lookups — independent of m , of $|A|$, of $|B|$, and of the run lengths. A run of any length is priced as a run of length one, because the kernel never looks inside it; symmetrically $\mathbf{R} \times \mathbf{R}$ by merge costs $O(r_A + r_B)$ with no index. This is what makes a run count a cost parameter and not merely a storage parameter, and it is the only bridge between the two here. It reaches *operation count*, not time: per-operation constants still differ across cells.

What randomness would predict. Let each position be set independently with probability d , and write $\kappa(\rho)$ for the number of stored items representation ρ must touch. The first two entries below are exact; the last two are the leading-order forms of exact Bernoulli expectations, written in $s = \min(d, 1 - d)$,

$$\mathbb{E} \kappa(\mathbf{B}) = \left\lceil \frac{m}{w} \right\rceil, \quad \mathbb{E} \kappa(\mathbf{S}) = md, \quad \mathbb{E} \kappa(\mathbf{R}) = md(1 - d) = ms(1 - s), \quad \mathbb{E} \kappa(\mathbf{W}) = \frac{m}{w} [1 - (1 - s)^{2w} - s^{2w}], \quad (5)$$

where $\kappa(\mathbf{R})$ counts maximal runs and $\kappa(\mathbf{W})$ literal words plus fill groups. With the complement view making the best of **S** and **C** cost ms , every entry is constant in s or strictly increasing on $[0, \frac{1}{2}]$, and each is symmetric about $d = \frac{1}{2}$ by construction: under this null model the U-shaped cost curve is a theorem — about the model and the bitmap’s flatness, not about any method. Randomness also gives run containers nothing, since $\frac{1}{2}ms \leq \mathbb{E} \kappa(\mathbf{R}) \leq ms + 1$, so choosing *among* $\{\mathbf{S}, \mathbf{C} \circ \mathbf{S}, \mathbf{R}, \mathbf{W}\}$ buys at most a constant on structureless data. The bitmap is the exception and the exception is the point: $\kappa(\mathbf{B})$ is $\lceil m/w \rceil$ at every density, so the spread across the full matrix is Eq. (3) and unbounded. It is the envelope, not each cell, that follows $\Theta(\min(m/w, ms))$.

The deviation, which is the contribution. Real data is not random, and the representations part company with Eq. (5) in a way density cannot express. At fixed density $d \leq \frac{1}{2}$ with $k = dm$, $\kappa(\mathbf{B})$ and $\kappa(\mathbf{S})$ are constants — functions of (m, k) alone — while $\kappa(\mathbf{R})$ ranges over every integer in $[1, k]$ and $\kappa(\mathbf{W})$ from $O(1)$ to $\Theta(\min(k, m/w))$; the extremes are $[0, k]$, one run, and $\{i \lfloor m/k \rfloor : 0 \leq i < k\}$, of cardinality exactly k and stride at least two, hence k of them. Concretely $A = [0, \frac{m}{2})$ and $B = [\frac{m}{4}, \frac{3m}{4})$ both sit at density one half, where Eq. (2) leaves $\frac{m}{2} + 1$ feasible answers and Eq. (5) predicts $m/4$ runs apiece; each in fact has one, and Eq. (4) answers the pair in two lookups where $\mathbf{B} \times \mathbf{B}$ spends $\lceil m/w \rceil$ word operations. Writing $\sigma(\rho, X)$ for predicted descriptor length at X ’s density over actual, σ is identically one for **B** and **S**, at least $\max(d, 1 - d + \frac{1}{m}) \geq \frac{1}{2}$ for **R**, and unbounded above: real rows can be arbitrarily cheaper than random rows of the same density and at most about twice as expensive.

One impossibility follows, exactly. A selector reading density — or cardinality, or any function of them — evaluates one fixed function on every row of that density, so by the range above it mis-predicts $\kappa(\mathbf{R})$ by an unbounded factor on some rows at every density; a fixed representation cannot exploit σ either, having committed before σ is knowable. That rules out a family of selectors without choosing among the survivors, and two further readings do not follow: whether a mis-predicting model costs more than selecting nothing depends on which cell it picks and that cell’s actual cost, which is measured (Fig. 5b), not derived; and a run or occupancy count is $O(1)$ metadata that sees σ perfectly and could feed a model. What is proved is that selection must be gated on a statistic that sees structure rather than on cardinality — structural summary or direct kernel measurement is settled below, by measurement.

The null model is a floor, and it is the same floor three times. Independent placement at fixed density is not a typical case to be corrected for; it is the *extremal* arrangement for every mechanism used here, and always in the same direction. For a representation, Eq. (5) gives $\mathbb{E} \kappa(\mathbf{R}) = ms(1 - s)$ against a maximum of $\min(k, m - k + 1)$ over arrangements, so the null sits within a factor of two of the worst achievable descriptor and $\sigma(\mathbf{R}, X) \geq \max(d, 1 - d + \frac{1}{m}) \geq \frac{1}{2}$ at every density. Throughout,

σ is defined against the *fixed-cardinality* null, $k(m-k+1)/m$ runs, which conditions on the observable k and is exact; it differs from the Bernoulli form of Eq. (5) by $O(1)$ and nothing below depends on the choice. For a zone map of bin width W_z , the occupied-bin fraction is $p = 1 - (1-d)^{W_z}$ under independence against $p = d$ when every run spans a bin, and $1 - (1-d)^{W_z} \geq d$ for all $d \in [0, 1]$ and $W_z \geq 1$, with equality only at the endpoints: scattering maximises occupancy, so the summary's null cost is an upper bound on its cost and a lower bound on its value. For the prefix filter of the thresholded mode, with prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$, the probability that two scattered prefixes share an element is $1 - e^{-y}$ in $y = p^2/m$ — so the entire mechanism depends on the operands through that one dimensionless quantity — and any ordering that concentrates rare elements into prefixes makes real prefixes collide less often than scattered ones of the same length.

The consequence is uniform and it is the reason selection is measured rather than modelled. In each case the null is what the mechanism is worth on data with no structure to find; real data deviates from it only in the mechanism's favour; the magnitude of that deviation is not a function of density and so is invisible to any cost model whose inputs are cardinalities; and it is exactly the quantity the mechanism converts into avoided work.

Where the headroom vanishes. At $s = \frac{1}{2}$ the interval has width $m/2$ and excludes nothing, the cheapest list-based cell spends $m/2$ probes against $\lceil m/w \rceil$ words, and Eq. (5) offers neither **R** nor **W** an advantage: Eq. (3) falls below one. In the generic case the mid-density band belongs to **B** $\times$ **B**, and a selector choosing it there is correct rather than defeated. The qualification is carried throughout — this bounds what cardinality determines absent structure, not what any kernel must spend, and the structured example above is a counterexample to the unqualified form. It is also why the zone map of the next subsection matters in the one band where cardinality is useless.

Why the two mechanisms are selected between rather than stacked. Composing a zone map with a representation is not the product of the two, because filtering *concentrates* what it passes on: conditioned on a bin of width W_z being occupied its expected local density is d/p , $p = 1 - (1-d)^{W_z}$, not d — no summary hands a kernel anything sparser than one bit per bin. Charging the summary pass c/W_z of a bitmap scan and taking the operands independent, the composed cost relative to a plain bitmap is

$$\frac{c}{W_z} + p_A p_B \kappa\left(\frac{d}{p}\right), \quad (6)$$

$\kappa \leq 1$ being the envelope of Eq. (5) normalised against the bitmap. The composition therefore never costs more than the summary alone, yet is floored at c/W_z , below which a sparse representation is cheaper outright — three regimes, the middle one exactly $xe^{-x} > c/w$ in $x = W_z d$ bits per bin, opening where the summary undercuts enumeration and closing where bins stop being empty. This is the analytic form of the gating result of Fig. 5c, and it models *work*: it bounds where a filter has work to remove and says nothing about what one costs when it removes nothing.

Each mechanism's liability is two-sided, for unrelated reasons at the two ends. A summary is indexed by the universe while every compressed representation is indexed by the data, so their costs must cross. Charging the summary c/W_z of a bitmap scan as in Eq. (6), and a one-word-per-element representation $w d$, the low crossing is

$$d_{\text{low}} = \frac{c}{w W_z} = 3.1 \times 10^{-5} \quad (c = 1, w = 64, W_z = 512), \quad (7)$$

below which reading the summary costs more than enumerating the operands outright: at $d = 10^{-6}$ it does $31 \times$ the work of enumerating them. At the other end $p_A p_B \rightarrow 1$: no bin is empty, the summary proves nothing, and only its own c/W_z remains. A run- or fill-based representation crosses at its own density, wherever its descriptor length falls below m/W_z , but that a crossing exists at all is generic. Prefix filtering has the same shape in $y = p^2/m$: for $y \ll 1$ the surviving fraction is y itself, so pruning improves quadratically as prefixes shorten, while for $y > 1$ prefixes collide by default and the index earns nothing. This is why no mechanism here is applied unconditionally, and why each gate is a two-sided test rather than a threshold.

A narrowed contract, and what two integers then settle. Every statement above returns the cardinality itself. A caller may instead ask a narrower question — report only the pairs with $J(A, B) \geq t$, or only those with $|A \cap B| \geq \tau$, or restrict attention to operands whose size falls in a band — and the narrowing is itself exploitable, because Eq. (1) already says what sizes permit. Since $|A \cap B| \leq \min(a, b)$ and $|A \cup B| = a + b - |A \cap B| \geq \max(a, b)$,

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \leq \frac{\min(a, b)}{\max(a, b)}, \quad (8)$$

so $J(A, B) \geq t$ requires $\min(a, b) \geq t \max(a, b)$, and $|A \cap B| \geq \tau$ requires $\min(a, b) \geq \tau$. Restricting the inputs to a size band is the same test applied before a pair is formed rather than after. The hypotheses are that $a, b > 0$ and that $t \in (0, 1]$; nothing

is assumed about arrangement, about m , or about how either operand is stored. Both conditions are necessary and neither is sufficient, so the test discards pairs and never reports one — the survivors still go to an exact kernel. What it costs is one comparison between two integers already held as metadata, and no element of either set is read. This is the Swamidass–Baldi pruning bound³ together with the size filter of the similarity-join literature^{1,2}, and it is restated here to place it against the mechanisms above rather than as a result of this paper.

What that removes, and why a wider corpus helps it. Model a corpus by drawing cardinalities independently with $\log a$ uniform on $[0, \ln K]$ — which is the density $\Pr(a) \propto 1/a$, and so is exactly the $1/i$ cardinality spectrum this paper’s benchmark protocol mandates (“Corpora, provenance and loaders”), not an approximation to it. Then $\min(a, b)/\max(a, b) \geq t$ is $|\log a - \log b| \leq \gamma$ with $\gamma = \ln(1/t)$, and for two independent uniforms on an interval of length $L = \ln K$,

$$\Pr[\text{pair can qualify}] = 1 - \left(1 - \frac{\ln(1/t)}{\ln K}\right)^2, \quad \ln(1/t) \leq \ln K, \quad (9)$$

with the absolute form $(1 - \ln \tau / \ln K)^2$. At $K = 10^4$ the surviving fraction is 0.1449 at $t = \frac{1}{2}$, 0.0479 at $t = 0.8$, 0.0227 at $t = 0.9$ and 0.0111 at $t = 0.95$; at $K = 10^3$ and $t = 0.9$ it is 0.0303. Equivalently, examining every pair does $6.9\times$, $20.9\times$, $44.0\times$, $90.0\times$ and $33.0\times$ the work of examining only the survivors. Eq. (9) agrees with a 10^5 -sample simulation to three decimals at each of those points. Expanding for small γ gives $2\gamma \int g^2$ for any density g of $\log a$, so the governing quantity is the *effective log-spread* $1/\int g^2$ of the size distribution, and Eq. (9) is its log-uniform case.

Two hypotheses carry real weight and are stated rather than buried. First, cardinalities are integers, and $a = b$ passes at every t ; the continuous form is therefore optimistic where sizes are small, and drawing integer cardinalities as the generator does leaves 0.0185 rather than 0.0111 surviving at $t = 0.95$, $K = 10^4$. Second, Eq. (9) decreases monotonically in K — at $t = 0.9$ the surviving fraction falls from 0.0452 at $K = 10^2$ to 0.0114 at $K = 10^8$ — but that improvement is a property of the log-uniform family, not of heavy tails generally. Under the alternative reading in which *ranked* sizes scale as $1/i$, a density $\propto 1/a^2$ whose log is exponential, the surviving fraction is exactly $1 - t$ independent of K : the effective log-spread saturates at two nats however far the sizes are spread, and so does the gain. The favourable scaling in K is therefore claimed for the spectrum this paper benchmarks and not for every skewed corpus.

The size bound does not multiply with the mechanisms below it. Acting at a coarser granularity is not sufficient for savings to compose. The size test decides whether a pair is formed at all, while representation choice and the zone map act inside a pair that is formed, so the granularity argument of Eq. (6) predicts a product — and the product is not achieved, for a reason that argument does not capture: both mechanisms read the same statistic. The survivors of Eq. (8) are the pairs with $a \approx b$, and every mechanism below is cheapest exactly when a and b are *dissimilar*, since its cost tracks $\min(a, b)$ against a fixed $\lceil m/w \rceil$. Conditioning raises the mean cost of a survivor by

$$\frac{\mathbb{E}[\min(a, b) \mid \min/\max \geq t]}{\mathbb{E} \min(a, b)} = \frac{L^2(K(1-t) - \gamma)}{\gamma(2L - \gamma)(K - L - 1)} \xrightarrow{t \rightarrow 1} \frac{\ln K}{2}, \quad (10)$$

and the composed work exceeds the product of the two savings by that same factor: 4.40 at $K = 10^4$, $t = 0.9$, against $\ln K/2 = 4.61$. Simulating the composition directly reproduces it — the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition in fact costs 0.151%, a shortfall of $3.69\times$ once cardinalities are integers — and permuting which pairs pass the size test, so that the two mechanisms read uncorrelated statistics at the same rates, restores the product to within 0.6%. The composition is still strongly worth taking: at those parameters it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does $660\times$ its work. What is refuted is only the product, and the general form is that mechanisms compose multiplicatively when they act on different waste *and* are driven by different operand statistics; sharing a statistic costs the composition Eq. (10) even across granularities. Values here are analytic, computed over the model above at $m = 10^6$, and count work rather than time.

Where avoidance is unavailable, throughput is what remains. The mid-band above is not only where these results stop applying; it is where a different optimization applies, and the two are complementary rather than competing. Where Eq. (3) falls below one and $p_A p_B \rightarrow 1$, the work $B \times B$ performs is, in the generic case, work that is going to be performed, and what remains to optimize is the per-cycle throughput of its $\lceil m/(wV) \rceil$ vector AND-plus-popcount operations — a quantity none of the results above speaks to, and one the popcount literature has already driven close to the hardware limit⁶. Eq. (3) and Eq. (6) locate the boundary between the two domains; they do not rank them.

Representations and the per-cell cost model

Each row — one set X_i over a shared universe of m bits — is stored in one of several representations, each with a distinct asymptotic cost for computing an intersection *cardinality*, $|X_i \cap X_j|$, rather than the intersection itself. We define each

representation below, together with the cost of the cell it forms when paired with itself or with another representation; every cost-model symbol used elsewhere in this paper is defined here and only here.

Four representations form the symmetric pairing matrix. A dense bitmap (**B**) stores m bits directly, one per universe position, and costs $\Theta(m)$ to intersect against another dense bitmap by AND and popcount, independent of either side’s cardinality. A sorted array (**S**) stores the set’s $|X|$ set positions as sorted integers and costs $\Theta(|X|)$ against a bitmap, since each element is resolved by a single random probe. A run container (**R**) stores the set as r maximal runs (a, b) of contiguous set bits and, against a bitmap fitted with a rank index (below), costs $\Theta(r)$ — independent of run length. A fill-compressed word stream (**W**, an EWAH-style encoding of literal words interleaved with run-length-coded fill words) costs $\Theta(\ell + f)$ against a bitmap, where ℓ is the literal-word count and f the fill-word count, and its fills are skipped in constant time using the same rank index used for **R**. A fifth view, the complement (**C**), does not add a representation to the symmetric matrix; it is a strategy, described in the next subsection, for reusing whichever of **B**, **S**, **R** or **W** is cheapest on a row’s complement when that row sits above density one-half.

The cost of each symmetric cell is, in the notation above: $\mathbf{B} \times \mathbf{B} = \Theta(m)$ (naive; made sub-linear by the zone-map filter below); $\mathbf{B} \times \mathbf{S} = \Theta(|S|)$; $\mathbf{B} \times \mathbf{R} = \Theta(r)$ with a rank index; $\mathbf{B} \times \mathbf{W} = \Theta(\ell + f)$; $\mathbf{S} \times \mathbf{S} = \Theta(|A| + |B|)$ by merge or $\Theta(|A| \log(|B|/|A|))$ by galloping search when $|A| \ll |B|$; $\mathbf{S} \times \mathbf{R} = \Theta(|S| + r)$ by merge or $\Theta(|S| \log r)$ by binary search into runs; $\mathbf{S} \times \mathbf{W} = \Theta(|S| + \ell + f)$; $\mathbf{R} \times \mathbf{R} = \Theta(r_A + r_B)$ by run merge; $\mathbf{R} \times \mathbf{W} = \Theta(r_A + (\ell + f)_B)$; and $\mathbf{W} \times \mathbf{W} = \Theta((\ell + f)_A + (\ell + f)_B)$. Every cell except $\mathbf{B} \times \mathbf{B}$ is sub-linear in the universe size m .

The pairing matrix and its cells

The paper reports the ten cells of the symmetric representation matrix formed from $\{\mathbf{B}, \mathbf{S}, \mathbf{R}, \mathbf{W}\}$: $\mathbf{B} \times \mathbf{B}$, $\mathbf{B} \times \mathbf{S}$, $\mathbf{B} \times \mathbf{R}$, $\mathbf{B} \times \mathbf{W}$, $\mathbf{S} \times \mathbf{S}$, $\mathbf{S} \times \mathbf{R}$, $\mathbf{S} \times \mathbf{W}$, $\mathbf{R} \times \mathbf{R}$, $\mathbf{R} \times \mathbf{W}$, $\mathbf{W} \times \mathbf{W}$. The complement strategy, $\mathbf{C} \times \mathbf{B}$, is an eleventh, asymmetric strategy rather than a twelfth symmetric cell: when a row’s density places it above one-half, its complement is computed and intersected using whichever of the ten cells above is cheapest for that complement, and the requested cardinality is recovered by inclusion–exclusion, $|A \cap B| = |B| - |A^c \cap B|$, where A^c is A ’s complement against the shared universe. This convention is used consistently throughout: any count of “ten” cells in this paper refers to the symmetric matrix over $\{\mathbf{B}, \mathbf{S}, \mathbf{R}, \mathbf{W}\}$, and $\mathbf{C} \times \mathbf{B}$ is reported separately as a strategy rather than folded into that count. Roaring is never constructed as a cell of this matrix; it is used throughout as an external baseline only (below).

Each cell is implemented in its own source file (`cell_bb.cpp`, `cell_bs.cpp`, `cell_br.cpp`, `cell_sparse.cpp`, `cell_wah.cpp`, `cell_comp.cpp`), against a shared representation layer (`storm_repr.{h, cpp}`: `BitmapView`, `ListView`, `RunView`, `EwahView`, `ComplementView`). Several of the asymmetric cells have more than one implemented strategy — for example $\mathbf{B} \times \mathbf{R}$ has a rank-indexed variant and a scalar per-run-scan variant — and each variant was raced against the independent oracle described below before being admitted to any comparison. The variant reported for a given cell in Results is the one selected as fastest for the density regime or corpus shape being reported, not a single fixed implementation chosen in advance; the full record of implemented variants, including those that did not win any regime, is not itemized in this paper and is available in the project repository.

Zone-map and rank-index construction

Two mechanisms let a kernel decide how much of a row it needs to visit, rather than visiting all of it faster. To decide how much of a dense row a pairing needs to scan, we used a *zone map*, a bitmap of coarser granularity in which bit k records whether bin k of the underlying row — a contiguous block of **[zone-map bin width in bits]** bits — contains any set bit. To answer a run’s or a fill’s contribution to a bitmap without touching the bits it spans, we used a *rank (prefix-popcount) index*, following the rank/select lineage of Jacobson, Clark and Vigna’s `rank9`^{14–16}.

The zone map is built by a single forward pass over the row, or is free at query time when the underlying storage already carries block-occupancy metadata, as columnar formats commonly do; its storage cost is **[zone-map storage cost as a fraction of the bitmap it summarizes]** of the bitmap it summarizes. For a pair, the bitwise AND of the two rows’ zone maps, $\text{occ}_A \wedge \text{occ}_B$, is itself a bitmap of $m/\text{[zone-map bin width in bits]}$ bits, and its popcount is the *exact* number of bins the pairing needs to visit, not an estimate: a popcount of zero proves the two rows disjoint without visiting either. The rank index is built by a single forward prefix-popcount pass and stores **[rank-index storage cost as a fraction of the bitmap it indexes]** of the bitmap it indexes; for a run or fill spanning positions $[a, b)$ on the indexed side, its contribution to the intersection is answered in constant time as $\text{rank}_B(b) - \text{rank}_B(a)$, the number of set bits in B up to b minus the number up to a — this formula is used, unchanged, by every cell that pairs a run- or fill-based representation against an indexed bitmap.

Neither mechanism is applied unconditionally. The zone-map filter is consulted only ahead of cells that would otherwise scan a dense representation in full ($\mathbf{B} \times \mathbf{B}$ and $\mathbf{B} \times \mathbf{W}$); it is not applied to $\mathbf{B} \times \mathbf{S}$ or $\mathbf{B} \times \mathbf{R}$, which are already work-proportional to the sparse side and for which a summary can remove nothing a direct probe does not already skip. Applied to a row whose bins are uniformly occupied, the filter finds nothing to skip and its cost is pure overhead; the policy that decides, per pair, whether the filter is worth consulting is described as a selection policy in the next subsection, and its necessity is established as a result rather than assumed as a procedure (Fig. 5c). This is a re-parameterization of established occupancy-summary mechanisms rather

than a new one: BitFunnel’s hierarchical block index¹⁰, WAH/EWAH’s own uniform-fill words, and Lucene’s sparse-block bitset all skip empty regions by a similar test; what is specific to this setting is applying the same test as an *exact* work-count for all-pairs intersection cardinality, where the summary’s own popcount is the number of bins that must be visited rather than a heuristic estimate of it.

Selection policies: per-pair, per-tile, and probe-and-commit

Selection — choosing which cell to run for a given pair — is distinct from the zone-map and rank-index mechanisms above, which decide how much of an already-chosen cell’s work can be skipped; the two are reported separately throughout. Three selection granularities were evaluated.

Under *per-pair* selection, precomputed $O(1)$ metadata for each row — cardinality, run count, and per-row zone-map bin-occupancy count — is consulted once for every pair (i, j) , and the cell predicted cheapest under the cost model above is chosen. Under *per-tile* selection, rows are first partitioned into tiles of **[tile width in rows]** rows by sorting or bucketing on density and run count, so that rows sharing similar characteristics fall into the same tile; one cell choice is then made per tile, from the same $O(1)$ metadata aggregated over the tile, and applied to every pair inside it, hoisting the decision out of the innermost loop. Under *probe-and-commit*, named here as the $\varepsilon \rightarrow 0$ special case of Micro Adaptivity in Vectorwise¹¹, a small number (**[count of probe pairs sampled per tile before committing]**) of a tile’s pairs are timed under each candidate cell before any decision is made, and the tile commits to whichever cell was empirically fastest on that sample for its remaining pairs. All three policies are evaluated against an explicit runtime budget of **[stated selection-cost budget as a percentage of total runtime, from the design target this paper adopts]** of total runtime; a policy that exceeds the budget is reported as failing it, not as a qualified pass.

Selection cost was measured by instrumenting the decision path itself separately from the kernel calls it dispatches to, so that the reported cost of choosing is not conflated with the cost of computing. *Regret* against a bucket oracle is reported alongside selection cost for every online policy: the oracle is the best cell for a given bucket of pairs chosen with full hindsight after every candidate cell has been timed on it, and a policy’s regret is its runtime on that bucket expressed relative to the oracle’s runtime on the same bucket. The same bucket oracle is used to report regret for an always-Bbaseline and for a closed-form per-pair cost model evaluated from the metadata above but without online timing, so that measured selection can be compared directly against both a no-selection baseline and a modelled-selection baseline on the same metric.

Two distinct oracles are reported in this paper and are never interchanged. The *bucket oracle* defined above operates at bucket granularity and is the denominator for every regret figure. The *best-cell oracle* reported as an upper-bound column in Table 1 is coarser: for a whole corpus it takes the single cell whose measured end-to-end runtime on that corpus is lowest, chosen after the fact and charged no selection cost. Because it may not vary its choice within a corpus, it is not a lower bound on the bucket oracle’s runtime; it is reported only as a bound on what a per-corpus selector could achieve, and the deployed-versus-oracle gap in Table 1 is regret against that bound alone.

Corpora, provenance and loaders

Three corpus families were used. The first is the set of corpora CRoaring’s own benchmark harness runs by default (the *real-roaring-datasets* collection), including *census1881*, *census-income*, *weather_sept_85*, *wikileaks-noquotes*, *uscensus2000* and the *dimension_0xx* family, together with the row-sorted (*_srt*) clustering variant of each that the original Roaring papers also report. The second is a set of larger modern graph corpora added beyond CRoaring’s default set, drawn from public network-graph collections (including *com-LiveJournal*, *com-Orkut* and *as-skitter*). The third is genomic: two population-scale variant call sets, USHER SARS-CoV-2 and gnomAD chromosome 21, both stored variant-major (one row per variant site, one bit per sample), and one haplotype-major encoding of 1000 Genomes Project phase 3 chromosome 20 (one row per haplotype, one bit per site) used specifically to demonstrate that storage layout, not the underlying data, determines whether a corpus lands in the sparse or the mid-density regime. Universe size and density are reported per corpus in Table 1 and are not duplicated in prose here; every corpus is drawn against the same competitively tuned CRoaring baseline used throughout Results.

As a loader-correctness check, the *census1881* loader was validated by reproducing the published corpus statistics reported for it (universe size and mean row cardinality) against the values reported in the Roaring bitmap literature¹², rather than trusting the loader’s own output uninspected.

Container census

The container counts in Table 2 are read from CRoaring’s own introspection API, *roaring_bitmap_statistics()*, and not inferred from our measurements. The census is taken after *run_optimize()* and *shrink_to_fit()*, so that it reports the containers the library settles on once it has had the opportunity to choose run containers, matching the tuned baseline every ratio in this paper is computed against. Four fields are recorded per corpus: *n_array_containers*, *n_bitset_containers*, *n_run_containers*, and the total container count. The threshold governing that choice is a

vendor fact rather than a measurement of ours: CRoaring stores a 2^{16} -bit chunk as an array container unless the chunk's own cardinality exceeds the compile-time constant `DEFAULT_MAX_SIZE`, whose value is **[value of DEFAULT_MAX_SIZE in the pinned CRoaring version]** (declared at `roaring.h:2486` and tested at `dispatch at roaring.h:5515` in the pinned version below). The same fixed commitment prices a run container against a bitset by iterating every run and charging for its length (`run_bitset_container_intersection_cardinality, roaring.c:10901`).

Benchmark protocol and cache-residency statement

Every reported cell variant, including CRoaring, was compiled with the same compiler and flags, allocated with the same alignment, and given the same warm-up treatment before timing, so that no comparison advantages one implementation by construction. CRoaring was tuned in good faith before every comparison: `run_optimize()` and `shrink_to_fit()` were called before timing began, matching what a competent user of the library would do, and every ratio against CRoaring reported in this paper is computed against that tuned baseline, not against CRoaring's untuned default construction.

Per-cell density and strategy sweeps (Figs. 2 and 3) raced every candidate variant over an identical, pre-generated pair list in an identical order, so that the cache state each variant encountered at call time was comparable across variants; each variant's differential correctness against the oracle (below) was checked before any of its timings were accepted. A warm-up pass of **[warm-up iteration count or criterion]** preceded timed repeats. Wall-clock time was measured with a monotonic per-run timer; where a hardware cycle count is reported, it is derived from wall time and a frequency measured on the host at run time rather than assumed from a nominal clock speed, and is labelled as a derived quantity rather than a hardware performance-counter reading. On Apple silicon hosts, benchmark threads were requested onto the performance core cluster to avoid the heterogeneous core migration that otherwise mixes two different microarchitectures' timings into one nominally single-threaded measurement.

The working-set-size sweep behind the residency comparison (Fig. 3b) varied corpus footprint from **[smallest working-set size in the sweep]** to **[largest working-set size in the sweep]** per host, and cache-residency boundaries (L1, L2 and shared/last-level cache) were identified from each host's own reported cache geometry rather than assumed to be uniform across hosts (below). Every throughput number reported anywhere in this paper is qualified by the cache-residency regime it was measured in, named once here by regime (L1-resident, L2-resident, DRAM-resident) so that Results and Discussion can use the regime name as shorthand rather than restate cache sizes at each mention.

Statistics and aggregation

Two different aggregation conventions were used, for two different kinds of comparison, and neither is substituted for the other anywhere in this paper. For controlled synthetic sweeps in which every variant races the identical pair list under identical conditions (the density sweep, the asymmetric-cell strategy comparison, the working-set-size sweep), the reported cost is the *minimum* over **[repeat count for synthetic per-cell sweeps]** repeats, on the standard microbenchmarking rationale that the minimum estimates the underlying cost with scheduler and system-noise inflation removed, while the mean instead reports how noisy the run happened to be. For the real-corpus comparison (Table 1), where repeats necessarily interleave across corpora and share a host with other system activity over a longer measurement window, the reported ratio is instead the *median* over **[repeat count for real-corpus comparisons]** interleaved repeats, reported together with the observed range; no per-corpus ratio in this paper is quoted to two significant figures without that range being available alongside it.

Run-to-run variance was largest for the corpora and hosts named explicitly in Results and in the notes to Table 1; any corpus whose repeats disagreed beyond **[stated dispersion tolerance used to flag a corpus as unquotable]** was excluded from headline ranges as unquotable, and named as excluded rather than silently dropped. On **[count of corpora, of the total corpus set, where the winning cell itself changed between repeats]** corpora, the winning cell reported in Table 1 itself changed between repeats; this is disclosed here once so that Table 1 can report a single, clearly labelled cell per corpus without re-explaining the variance underlying it at every mention.

Correctness oracle and differential testing

Every cell variant is checked against an independent scalar oracle (`oracle.cpp`) that computes intersection cardinality by direct bit-by-bit or element-by-element counting, sharing no code path with any of the variants under test, before that variant is admitted to any timed comparison; a variant that returns a wrong cardinality is not reported as fast regardless of its measured speed. Correctness is tracked at three levels: an ABI-level regression suite (`test_storm.c`, **[test_storm check count]** checks against the oracle, **[test_storm failure count, expected "0"]** failures) that is compiled and linked as C to serve as the public C ABI's own regression test; a cell-level differential suite (`test_cells`, **[test_cells check count]** checks, **[test_cells failure count, expected "0"]** failures) that exercises every implemented cell variant against the oracle across a grid of densities, structures and alignments; and a larger differential run, on the order of **[cross-architecture differential check count per host family]**, executed once per host family during the cross-architecture measurement campaign. The public C ABI surface was verified independently with `nm`: **[count of unmangled STORM_-prefixed exported symbols]** unmangled `STORM_` symbols

and **[count of defined mangled (C++) symbols, expected "0"]** defined mangled symbols, confirming that the public interface is C-linkable as declared.

The cell-level suite is driven by a fuzz grid over {row count, word count, density, within-row structure (uniform, clustered, run-based), alignment}, following the same axes used for every kernel change in this project. The regression suite's own ability to fail was itself verified rather than assumed: each of a set of known defects fixed earlier in the project was reverted individually and the suite was confirmed to fail before being re-fixed, establishing that a red result from the suite is informative and not merely a suite that always passes.

Hardware and toolchain

Measurements were taken on **[count of microarchitectures used across the whole paper, expected "four"]** microarchitectures spanning two independently hand-written SIMD kernel paths — one using ARM NEON, the other using x86-64 AVX-512 with the VPOPCNTDQ extension — plus a portable scalar fallback used where neither hand-written path applies. No architecture-specific SVE or SVE2 kernel exists in this project's kernel sources at the time of writing: the Arm Neoverse-class hosts used here run the generic NEON path, on hardware that is SVE2-capable but for which no SVE2-specific intrinsic code was written, and no result in this paper should be read as evidence about an SVE2 kernel. The same applies to AVX2 and SSE4.2: build options exist to gate an older, separate code path (described below) to those instruction sets, but no pairing-matrix cell was written against them.

Each host is identified by **[per-host microarchitecture name, core/cluster configuration, and nominal clock speed]** and by its cache hierarchy (**[per-host L1d, L2, and shared/last-level cache sizes]**), cross-referenced against the cache-residency regimes named above rather than restated per measurement. All code was compiled with **[compiler name and version per host]**, using **[compiler flags used, and whether native-architecture dispatch (`-march=native` or equivalent) was enabled or disabled for a given reported number]**. The CRoaring baseline used throughout is pinned at version **[CRoaring version pin, expected "v4.7.2" pending re-verification against current master]**, vendored at `third_party/croaring_amalg`, so that the exact baseline is reproducible rather than referred to generically as "CRoaring."

Data and code availability

The benchmark harness, kernel sources, and per-corpus loader scripts described above are available in the project repository under the Apache License, Version 2.0. The pairing matrix, the selector and the tile pipeline described in this paper are not, at the time of writing, exposed through the project's public C ABI; the public ABI currently surfaces an earlier, separate code path not used for any measurement reported here. This paper reports a systematic measurement of the pairing-matrix mechanism as implemented in the project's benchmark and kernel sources, not a released, ABI-stable tool, and this is stated here as a fact about the present state of the code rather than a property that is apologized for. Public real-world corpora were obtained from their original sources and reproduced locally with the loader scripts referenced above; raw per-corpus timing data, including the repeats and dispersion referenced in Statistics and aggregation, is deposited as Supplementary material.

References

1. Bayardo, R. J., Ma, Y. & Srikant, R. Scaling up all pairs similarity search. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*, 131–140, DOI: [10.1145/1242572.1242591](https://doi.org/10.1145/1242572.1242591) (2007).
2. Xiao, C., Wang, W., Lin, X. & Yu, J. X. Efficient similarity joins for near duplicate detection. In *Proceedings of the 17th International Conference on World Wide Web (WWW)*, 131–140, DOI: [10.1145/1367497.1367516](https://doi.org/10.1145/1367497.1367516) (2008).
3. Swamidass, S. J. & Baldi, P. Bounds and algorithms for fast exact searches of chemical fingerprints in linear and sublinear time. *J. Chem. Inf. Model.* **47**, 302–317, DOI: [10.1021/ci600358f](https://doi.org/10.1021/ci600358f) (2007).
4. Haque, I. S., Pande, V. S. & Walters, W. P. Anatomy of high-performance 2D similarity calculations. *J. Chem. Inf. Model.* **51**, 2345–2351, DOI: [10.1021/ci200235e](https://doi.org/10.1021/ci200235e) (2011).
5. Dalke, A. The chemfp project. *J. Cheminformatics* **11**, 76, DOI: [10.1186/s13321-019-0398-8](https://doi.org/10.1186/s13321-019-0398-8) (2019).
6. Muła, W., Kurz, N. & Lemire, D. Faster population counts using AVX2 instructions. *The Comput. J.* **61**, 111–120, DOI: [10.1093/comjnl/bxx046](https://doi.org/10.1093/comjnl/bxx046) (2018). ArXiv:1611.07612.
7. Fu, Y.-X. Statistical properties of segregating sites. *Theor. Popul. Biol.* **48**, 172–197, DOI: [10.1006/tpbi.1995.1025](https://doi.org/10.1006/tpbi.1995.1025) (1995).
8. Chambi, S., Lemire, D., Kaser, O. & Godin, R. Better bitmap performance with Roaring bitmaps. *Software: Pract. Exp.* **46**, 709–719, DOI: [10.1002/spe.2325](https://doi.org/10.1002/spe.2325) (2016). ArXiv:1402.6407.
9. Lemire, D. *et al.* Roaring bitmaps: Implementation of an optimized software library. *Software: Pract. Exp.* **48**, 867–895, DOI: [10.1002/spe.2560](https://doi.org/10.1002/spe.2560) (2018). ArXiv:1709.07821.

10. Goodwin, B. *et al.* BitFunnel: Revisiting signatures for search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 605–614, DOI: [10.1145/3077136.3080789](https://doi.org/10.1145/3077136.3080789) (2017).
11. Răducanu, B., Boncz, P. & Żukowski, M. Micro adaptivity in Vectorwise. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 1231–1242, DOI: [10.1145/2463676.2465292](https://doi.org/10.1145/2463676.2465292) (2013).
12. Lemire, D., Ssi-Yan-Kai, G. & Kaser, O. Consistently faster and smaller compressed bitmaps with Roaring. *Software: Pract. Exp.* **46**, 1547–1569, DOI: [10.1002/spe.2402](https://doi.org/10.1002/spe.2402) (2016). ArXiv:1603.06549.
13. Klarqvist, M. D. R. Tomahawk: Fast calculation of LD and genotype correlation in large-scale cohorts. <https://github.com/mklarqvist/Tomahawk> (2017). Software repository, created 2017-07-17.
14. Jacobson, G. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, 549–554, DOI: [10.1109/SFCS.1989.63533](https://doi.org/10.1109/SFCS.1989.63533) (1989).
15. Clark, D. R. *Compact Pat Trees*. Ph.D. thesis, University of Waterloo (1996).
16. Vigna, S. Broadword implementation of rank/select queries. In *Experimental Algorithms (WEA 2008)*, vol. 5038 of *Lecture Notes in Computer Science*, 154–168, DOI: [10.1007/978-3-540-68552-4_12](https://doi.org/10.1007/978-3-540-68552-4_12) (Springer, 2008).

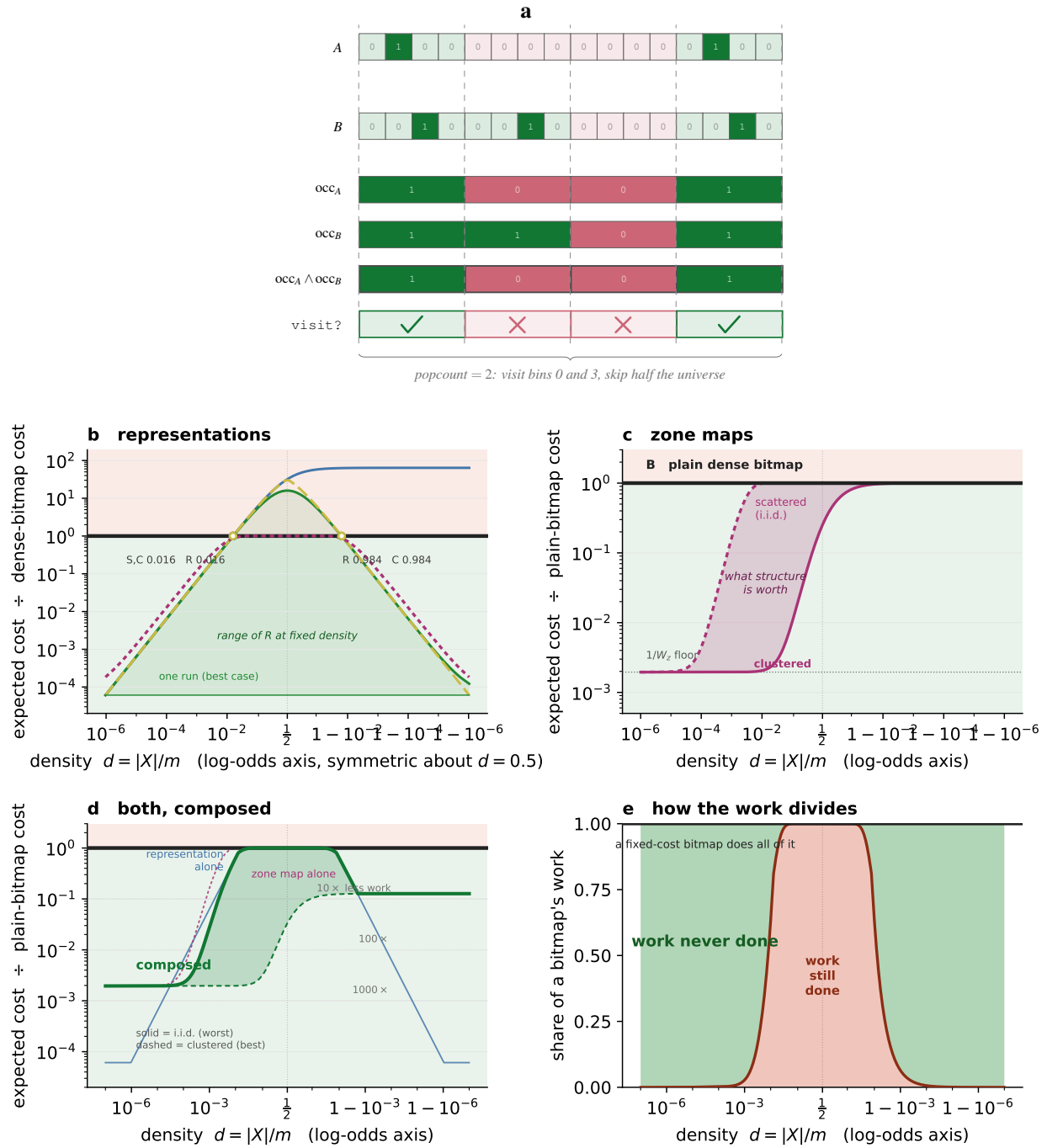


Figure 2. What a fixed cost leaves on the table, analytically. All panels are derived, not measured, and model *work* rather than time. **a**, How a zone map works, within one pair. Each bin of positions is summarised by one bit recording whether that operand holds anything in it. Colour encodes occupancy and each row carries its own: green where the row occupies the bin, red where it does not. $\text{occ}_A \wedge \text{occ}_B$ is the bitwise AND of the two summaries — the mechanism itself — and *visit?* reads it out. Bin 1 is the case to follow: *B* occupies it and is green, *A* does not and is red, so the AND is zero and the pair skips the bin even though one side holds data there. The AND costs a fraction of the rows it summarises and its popcount is the *exact* number of bins to visit, not an estimate; a popcount of zero proves the operands disjoint without reading either. **b**, Expected cost of each representation for a uniformly random set, as a multiple of the dense-bitmap cost, on a log-odds density axis. Every cost is proportional to the universe m , so this panel is scale-invariant and the crossovers are universal constants: open circles at $d = 1/w = 0.0156$ for S and C, 0.0159 for R, and their complements. The bitmap is the line at 1. Shading marks the range of R at fixed density — one run at best, the i.i.d. value at worst — which is the deviation the method exploits. **c**, The same for a zone map, whose cost relative to a plain bitmap pairing is $1/W_z + p_A p_B$ with $p = 1 - (1 - d)^{W_z}$. Arrangement, not density, sets p : scattering gives $p = 1 - (1 - d)^{W_z}$ and clustering $p = d$, and at $d = 0.01$, $W_z = 512$ those differ by $482\times$. Scattering is the worst case, so the i.i.d. curve is a floor on what a summary is worth. **d**, The two composed. Filtering concentrates density — an occupied bin has local density d/p — so the representation used inside surviving bins is the one for d/p . Decade rules mark depth: in the clustered case the composition costs $1/W_z$ of a bitmap across the sparse range (0.195% at $W_z = 512$) and decays only in the dense half, to 12.7%. The floor is $1/W_z$ and the knee sits at $d = 1/8$ for every bin width. **e**, The same as a division

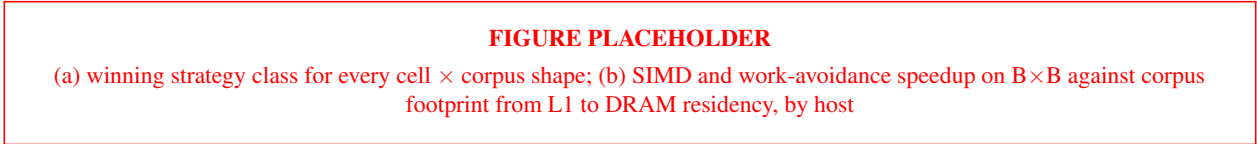


Figure 3. Vectorization’s advantage decays with working-set size; work avoidance does not. **a**, Strategy map: for each pairing cell (columns) at each corpus shape (rows), the class of the winning variant — work avoidance (rank lookup, zone-map skip or search-strategy choice) versus SIMD throughput — over [total asymmetric measurement points] measured cell-shape points. $B \times B$ is the only cell in which a vectorized variant is available to win at all. **b**, Speedup over a common scalar $B \times B$ baseline against corpus footprint, on [host count in the working-set sweep] hosts spanning [list of ISAs covered by the working-set sweep]: a SIMD dense-bitmap variant (solid) and a zone-map-planned variant of the same cell (dashed), so the two series differ only in mechanism. Line style encodes mechanism, colour encodes host. Vertical guides mark the boundary between L2-, last-level-cache- and DRAM-resident footprints on [reference host for the cache-boundary guides]; the footprint-to-cache-level mapping differs by host (Methods, “Benchmark protocol and cache-residency statement”). Horizontal dashed line at $1.0\times$ (no benefit over scalar). Corpus shape held fixed at [corpus shape held fixed across the working-set sweep] throughout **b**.

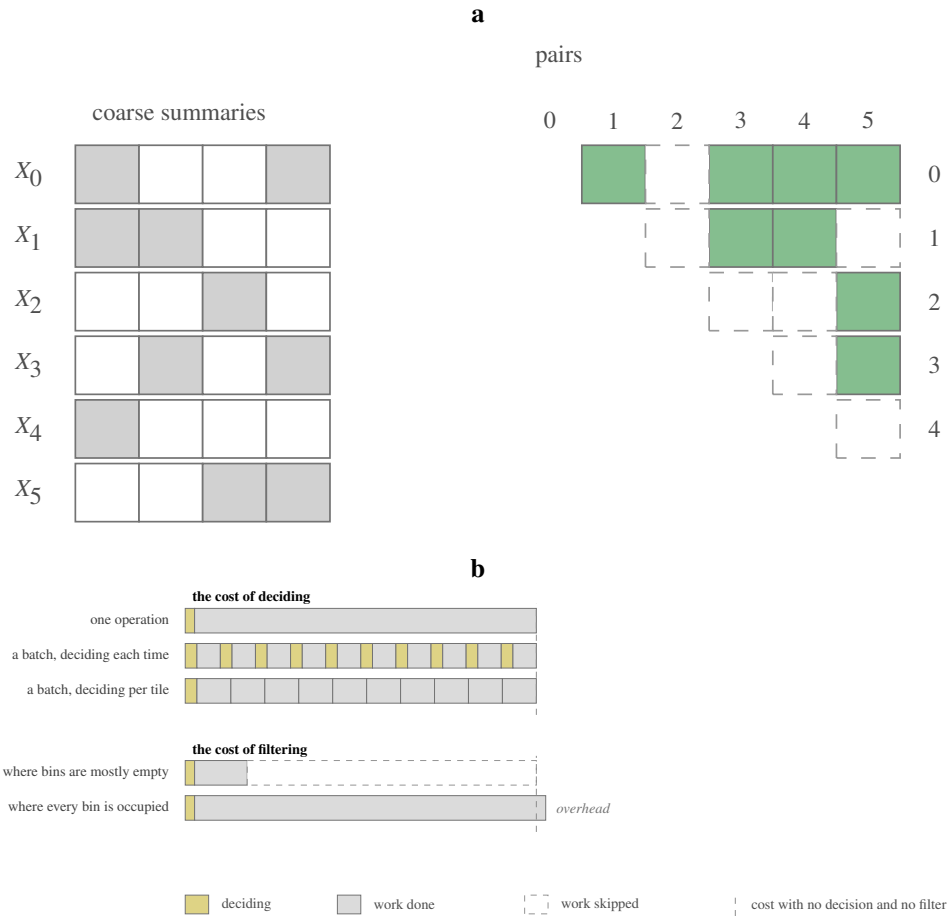


Figure 4. The batch regime: rejecting pairs wholesale, and paying for a decision once. Both mechanisms here need many operations rather than one, and what they remove is repetition. **a**, An overlap pre-filter across a set of operands. The same occupancy idea as a zone map (Fig. 2a) but at a coarser, chosen width and computed once per set, then ANDed pairwise: shaded pair cells may overlap and are passed to a kernel, dashed cells are provably disjoint and are answered without touching data. One pass over the summaries settles most of the pair matrix. Note the distinction from prefix filtering (Fig. 6), which is candidate *generation* and never enumerates the pair set at all. **b**, Why the cost of a decision depends on what it governs. Each bar spends left-to-right through one unit of useful work; the dashed rule marks that work with neither a decision nor a filter. *Deciding*: a single operation absorbs the decision easily, since it is read from $O(1)$ metadata and the operation it governs is large; the same decision repeated for every operation of a batch is a large fraction of the total, because each operation there is small; and hoisting it to a tile — deciding once, then committing — restores the single-operation picture. *Filtering*: where bins are mostly empty a summary converts most of the work into skipped work; where every bin is occupied it finds nothing to skip and its cost carries the bar past the rule. The cost of deciding therefore scales inversely with how large the operation being decided about is, which is why the two regimes need different policies and why every mechanism is gated rather than always on. Schematic; no measured data.

FIGURE PLACEHOLDER

Selection cost against the runtime budget, regret against a bucket oracle, and gated-vs-unconditional filter speedup, by policy and host

Figure 5. Selection is cheap only when hoisted, and work avoidance must itself be gated. **a**, Selection cost as a percentage of total runtime for three policies – per-pair, per-tile, and probe-and-commit – on [host count in the selection-cost sweep] hosts. Dashed line marks the [selection-cost budget, as a fraction of runtime] runtime budget; hatched bars fail it. **b**, Regret of each policy against a bucket oracle that times every candidate cell in bulk over pairs grouped by shape and keeps the per-bucket minimum – an unattainable, non-deployable denominator, and a measured lower bound on true per-pair regret (Methods). Hatched bars (all-bitmap; the per-pair cost model) are shown for scale, not as deployment candidates. **c**, Speedup of the zone-map work-avoidance filter over the unfiltered kernel across [count of corpora in the gating comparison] corpora (sorted by unconditional-filter speedup), applied unconditionally (grey) versus gated by the same selection machinery as **a** (green). Dashed line at $1.0\times$; annotated counts give the number of corpora harmed under each policy.

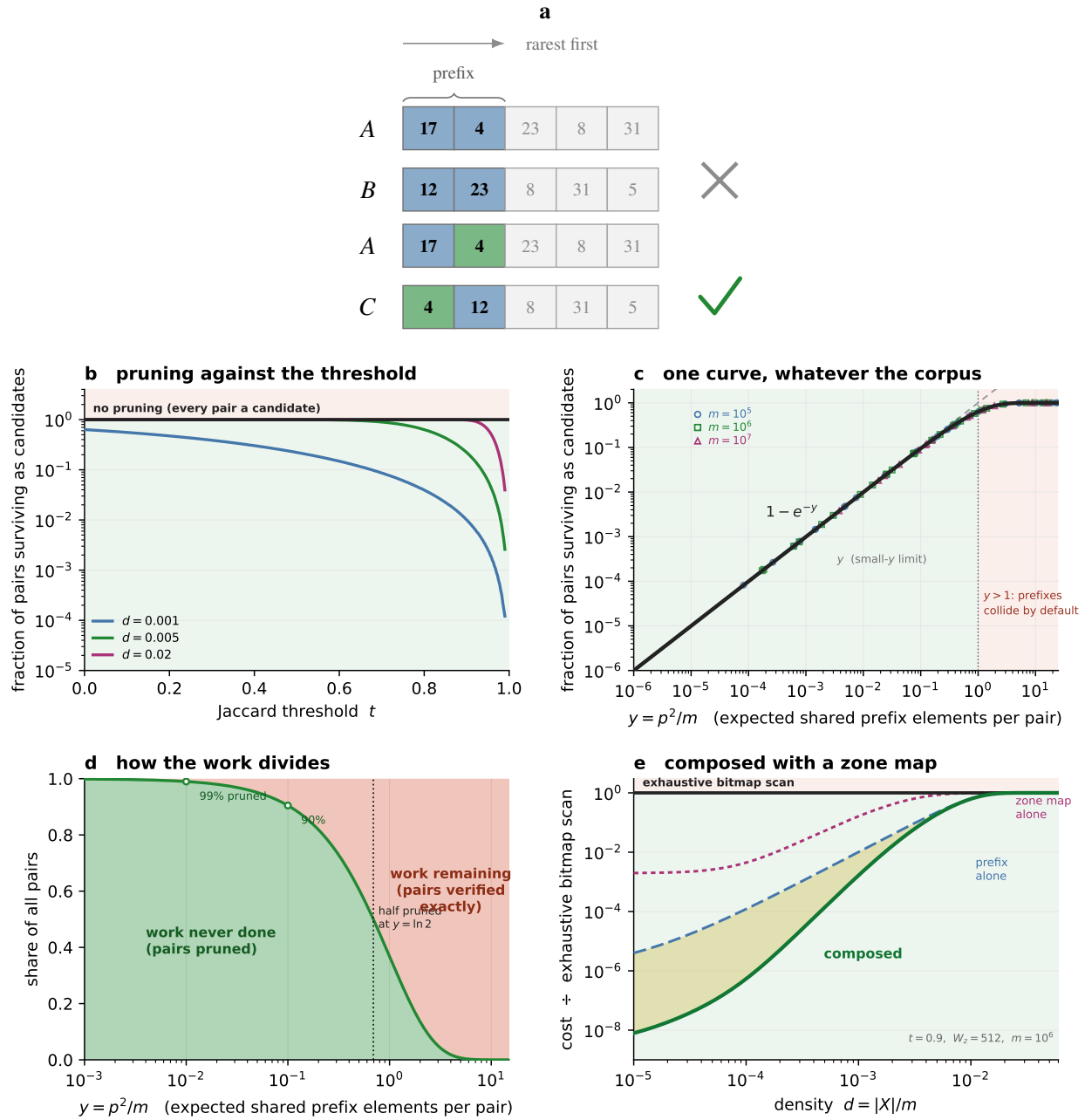


Figure 6. Prefix filtering, analysed on the same terms — and reducible to a single variable. Prefix filtering serves the opt-in thresholded mode, in which only pairs of Jaccard similarity at least t are reported. With elements in a global order and prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$, any qualifying pair must share an element between its prefixes, so indexing prefixes alone answers the thresholded question exactly. Unlike the mechanisms of Fig. 1c–d it is candidate *generation*: the quadratic pair set is never enumerated. **a**, How the rule works. Elements are laid out in a global order — by document frequency, rarest first, hence the non-monotonic labels — and each set’s prefix (saturated, bold) is its rarest $p(X)$ elements, a length that depends on t ; the remainder is greyed. Any pair with $J(A, B) \geq t$ must share at least one element between its prefixes. Top: the prefixes have no element in common, so the pair is discarded without computing any intersection. Bottom: one element (green) appears in both, so the pair becomes a candidate and is verified exactly. **b**, Fraction of pairs surviving as candidates against the threshold, at universe 10^6 . Pruning is governed by how long the prefixes are, and prefix length falls as t rises, so the same filter that removes almost everything at a high threshold removes almost nothing at a low one — and denser operands need a higher threshold to reach the same pruning. **c**, All of that collapses onto one curve. For prefixes of length p over a universe of m the expected number of shared prefix elements per pair is $y = p^2/m$, and the surviving fraction is exactly $1 - e^{-y}$ for scattered operands; markers are (d, t, m) combinations drawn at random across three universe sizes, and they lie on the curve by construction rather than by fit. Two regimes follow. For $y \ll 1$ the surviving fraction is y itself, so pruning improves *quadratically* as prefixes shorten. For $y > 1$ prefixes collide by default, essentially every pair is a candidate, and the index is pure overhead — the same shape of liability as a summary too sparse to pay for itself (Fig. 2). The useful operating regime is therefore $y \ll 1$, and y — not t , and not density — is the quantity a selector should test. **d**, The same information as a division of labour: on a linear axis, the share of all pairs never examined against the share surviving to exact verification. The two are equal at $y = \ln 2$. **e**, Prefix filtering composed with a zone map. Unlike the two mechanisms of Fig. 2c, these *do* compose —